澳大利亚国立大学

计算机学院

JHQ-GPU

面向表示的 GPU 近似最近邻搜索与工作负载级精排预算校准

Rui Luo

导师：Dr Mengxuan Zhang

中文阅读对照稿

章节、公式与图表编号对应英文稿。图内保留英文标签，便于与原始实验及英文稿核对。

COMP8800 — Advanced Computing Research Project

2026 年 10 月

摘要

近似最近邻（ANN）搜索支持大规模嵌入向量集合上的相似性检索。GPU 可以加速搜索，但有限显存使压缩索引具有吸引力。JHQ 是一种层级量化方法：先用紧凑编码筛选候选，再利用残差编码对较小的候选集精排。本文开发 JHQ-GPU，研究如何利用这种表示实现高效的 GPU 建库与搜索。

实现将 GPU 模型拟合和分批编码，与有界输入暂存、打包且按倒排列表排列的布局结合。扫描将每张 256 项距离表替换为两张 16 项表，保持实数算术下的分数。分组扫描在固定的已计算分数与位置排序下，保持精确的全局候选选择。校准通过比较当前结果与较大预算参考返回的近邻，在无需真值标签的情况下决定精排候选数量。

在 NVIDIA RTX 5090 上，跨方法评测使用 Vogue、arXiv、OpenAI-1536 和 OpenAI-3072。在召回率 0.97、batch 1,000 下，JHQ-GPU 在这四个数据集上的主机到主机吞吐为图索引 CAGRA fp32 的 1.32–1.75×，并在其中三个数据集上取得已测方法的最高吞吐。对于更大规模的 BGE 和 Stella 文本嵌入数据集，原始 FP32 向量的存储需求已超出设备显存，JHQ-GPU 仍可在单张 GPU 上完成搜索；最大索引包含 17.78 百万个向量，常驻 GPU 显存为 20.44 GiB。

致谢

感谢导师 Mengxuan Zhang 博士提出研究方向，并在项目过程中通过定期讨论提供指导。

研究与 AI 辅助

我使用 OpenAI Codex 辅助实质性起草、修改、LaTeX、分析及代码 / 日志核查 [30]；使用 ChatGPT 辅助规划与审核 [29]；使用 Anthropic Claude 辅助实现、实验及审核 [4]。选定提示词与使用详情见AI 使用记录。我对本论文中的主张负责。

目录

摘要 i

致谢 i

研究与 AI 辅助 i

符号与缩略语 vi

1 引言 1
- 1.1 背景与研究动机 . . . 1
- 1.2 问题定义与研究范围 . . . 1
- 1.3 技术挑战 . . . 1
- 1.4 研究问题 . . . 2
- 1.5 研究贡献 . . . 2

2 背景与相关工作 3
- 2.1 距离与召回率 . . . 3
- 2.2 IVF、PQ、ADC 与精排 . . . 3
- 2.3 JHQ 表示及其假设 . . . 4
- 2.4 GPU 执行概念 . . . 5
- 2.5 基于图的 ANN 索引 . . . 5
- 2.6 压缩搜索与自适应精排 . . . 5
- 2.7 与已有方法的关系 . . . 6

3 研究设计与测量 7
- 3.1 实验比较与控制变量 . . . 7
- 3.2 数据集、查询、真实近邻与划分 . . . 7
- 3.3 计时与显存测量 . . . 8
- 3.4 基线与前沿构造 . . . 9
- 3.5 实验记录与可复现性 . . . 10

4 GPU 索引设计与构建 11
- 4.1 构建与搜索流程 . . . 11
- 4.2 训练、旋转与码本生成 . . . 11
- 4.3 IVF 分配、ID 与物理内存布局 . . . 13
- 4.4 常驻存储、工作区与索引生命周期 . . . 15
- 4.5 构建成本与已测规模 . . . 16
- 4.6 索引表示与验证 . . . 17

5 面向表示的 GPU 搜索 19
- 5.1 查询流程与排序分数 . . . 19
- 5.2 精确的 Cartesian LUT 分解 . . . 20
- 5.3 打包编码与候选并行内存访问 . . . 22
- 5.4 多 block 扫描中的精确全局主码 Top-c . . . 23
- 5.5 共享完整键阈值与执行调度 . . . 26
- 5.6 Lane 协作残差评分与最终 Top-k . . . 28
- 5.7 IVF 探测列表选择 . . . 29

6 工作负载级预算校准 31
- 6.1 精排预算与搜索成本 . . . 31
- 6.2 输出一致性与预算选择 . . . 31
- 6.3 预算搜索与单调性条件 . . . 33
- 6.4 预算选择基线 . . . 34

6.5 校准质量与复用成本 35
6.6 校准复用与适用范围 35

7 实验评估 **36**
7.1 实验协议 36
7.2 跨系统召回率与吞吐比较 36
7.3 Cartesian LUT、打包载入与 Lane 精排 39
7.4 选择正确性与性能 41
7.5 留出预算质量、吞吐与回本 42
7.6 批量大小、调度与负载不均 45
7.7 冷构建、常驻显存与已测规模 46
7.8 数值检查与召回分析 49

8 讨论与未来工作 **51**
8.1 设计启示 51
8.2 替代评分与编码实验 51
8.3 内部与外部有效性的威胁 52
8.4 未来研究方向 53
8.5 部署建议 53

9 结论 **54**
9.1 对四个研究问题的回答 54
9.2 总结与展望 54

参考文献 **55**

插图目录

2.1 候选与评分级联 4
3.1 主机与设备计时边界 8
4.1 构建与查询架构 11
4.2 GPU 索引的物理布局 14
4.3 一遍与两遍构建 16
4.4 独立的构建分配生命周期 17
5.1 查询阶段及其选择边界 19
5.2 Cartesian 主码 LUT 分解 21
5.3 候选并行的打包访问 22
5.4 局部覆盖与截断成对归并 25
5.5 共享完整键过滤 27
5.6 十六 lane 残差精排 29
6.1 受精排预算影响的工作 31
6.2 工作负载级校准与评估 32
6.3 嵌套与非嵌套候选池 33
7.1 E1 召回率—吞吐前沿 37
7.2 TF32 与 FP32 精度前沿 38
7.3 小 Batch 竞争区间 39
7.4 Cartesian 查找表分组 40
7.5 打包访问与精排对照 40
7.6 不同样本量下的留出风险 43
7.7 有限池风险预测与观测 44
7.8 校准后复用的直接累计时间 45
7.9 依赖 Batch 的执行区间 46
7.10 记录的构建成本 47
7.11 常驻显存分解 48
7.12 大数据集搜索前沿 49
8.1 Mailman 与面向排名的编码 52

表格目录

1.1 技术贡献与支持证据 2
2.1 相关方法与 JHQ-GPU 设计问题 6
3.1 研究问题与实验对照 7
3.2 数据集与训练配置 7
3.3 E1 主要调参维度 9
3.4 组件实现 10
4.1 每向量存储模型 15
4.2 各构建阶段的行数组占用模型 16
4.3 原始输入的 FP64 重编码审计 18
4.4 表示约束与审计覆盖 18
5.1 查询阶段的选择约束 20
5.2 Cartesian 查找表的权衡 21
6.1 预算对照及其作用 34
7.1 召回率 0.97 时的跨系统比较 36
7.2 匹配配置下的 FP32 敏感性 37
7.3 同会话精度锚点 38
7.4 小 Batch 匹配召回比 39
7.5 128 probes 下的精排收益 41
7.6 严格计算分数选择检查 41
7.7 Probe 选择吞吐比 42
7.8 全池预算校准 43
7.9 抽样预算的留出质量 43
7.10 抽样校准经济性 44
7.11 500 个复用 Batch 的直接成本 45
7.12 集成执行与调度收益 46
7.13 冷构建与常驻显存 47
7.14 E1 分配范围 48
8.1 有效性威胁 52

符号与缩略语

符号	含义
N, d	数据库向量数与原始维度
M, D_s	主码子空间数与每子空间维度
$a_{\text{add}}, a_{\text{in}}, a_{\text{assign}}$	配置 add 行数、输入暂存行数及分配评分缓冲的独立容量（行数）
n_b, S_{stage}	流水缓冲数及每个输入 / 旋转数组的字节预算
Π	方形正交变换；$q' = \Pi q$
B, B_r	每主码子空间位数与每残差坐标位数
$\rho(p), o_\ell$	物理位置 p 的输入 ID，及 IVF 列表 ℓ 的起点
k	返回近邻数
n_c, c	路由候选数及接受精排的主码保留数
α	精排倍率，$c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	样本数；允许缺失的参考槽位数
$C_\alpha, \mathcal{R}_\alpha$	有效保留集合；预算 α 下返回的 ID
$\alpha_{\text{ref}}, B_{\text{reuse}}$	参考精排预算与后续复用 batch 数
g_{LUT}	分解查找表的组数
g_{scan}	每查询扫描组数；零表示单 block 路径
$C_{\text{buf}}, T_{\text{blk}}$	缓冲容量；每 block 线程数
$n_{\text{LUT}}(g_{\text{LUT}})$	每主码子空间 LUT 项数
$\kappa(D, p)$	阈值使用的打包完整键
ℓ_{ref}	每个保留候选的协作 lanes 数
D_{raw}	$\|q - x\|_2^2$
D_{P}	$\|q' - \widehat{p}\|_2^2$
D_{H}	$\|q' - \widehat{p} - \widehat{r}\|_2^2 + \|q'\|_2^2$

缩略语	含义
ANN	近似最近邻
ADC	非对称距离计算
IVF	倒排文件
JHQ	Johnson–Lindenstrauss 增强的层级量化
LUT	查找表
PQ	乘积量化
QPS	每秒查询数
GT	真实近邻
FP32	IEEE binary32；GEMM 模式另行说明
TF32	低精度 Tensor Core 输入；FP32 累加
CV	$\text{sd}(n_c) / \text{mean}(n_c)$

第 1 章引言

1.1 背景与研究动机

向量检索从大型数据库中返回距离查询较近的嵌入向量。GPU 并行计算可以加速距离计算，但原始高维向量可能超出设备显存。压缩可以使更大的索引常驻 GPU，但搜索仍需读取编码并选择有希望的向量。JHQ 是一种层级量化方法，先比较紧凑的主码形成候选短名单，再用残差编码进行更详细的比较 [16]。本文研究如何在遵循这些阶段的评分与选择规则的同时，在 GPU 上高效执行它们，并测量质量与速度之间的权衡。

基于图的 GPU 索引存储向量和图连接；倒排文件（inverted file，IVF）索引则存储在选定数据分区中搜索的向量表示 [18, 28, 33]。这些结构若超出设备显存，完整索引便无法常驻。本文最大的基准数据集 Stella 包含 17.78 百万个 1,024 维文本嵌入向量。其原始 FP32 存储需求、设备可用显存和 JHQ-GPU 常驻分配量，依次说明了容量约束（第7.7节）：

$$S_{\text{raw}} = 67.81\,\text{GiB} > S_{\text{GPU}} \simeq 31.8\,\text{GiB} > S_{\text{resident}} = 20.44\,\text{GiB}.$$

这说明有必要构建常驻 GPU 的压缩索引，直接搜索编码，而非在显存中保存完整的 FP32 数据库。

1.2 问题定义与研究范围

给定向量数据库 $\mathcal{X} \subset \mathbb{R}^d$ 和查询 q，搜索任务是在平方欧氏距离下返回 k 个近似最近邻。本文的研究问题是：如何在单张 GPU 的显存约束内构建并搜索 JHQ 压缩索引，以及如何在给定召回率目标下提高查询吞吐。

搜索先选择相关的 IVF 列表，使用主码粗筛其中的向量，再用残差编码对较小的候选集重新评分。精排预算控制进入最后阶段的候选数量。真值（ground truth，GT）给出原始向量空间中的精确近邻；召回率是其中被找回的比例。

本文保留 JHQ 不改变维度的正交变换和层级编码（第2.3节），研究 GPU 构建、数据布局、搜索执行和精排预算选择。校准不使用 GT，而是比较较小预算与较大预算参考返回的结果，再为固定索引和假设稳定的查询分布复用一个选定倍率。最终召回率通过 GT 评测核查。

评测使用一张 RTX 5090，执行平方 L2 距离的 top-10 搜索。主要跨系统比较使用四个数据集，batch 为 1,000；查找表、内存布局和预算实验覆盖六个数据集。一项小批量研究增加 batch 8/32（第7.2.6节）。索引和核心搜索计算常驻 GPU；计时包含搜索调用内部的查询准备与传输。实现范围不含动态更新、主机分页和多 GPU 执行；十亿规模搜索尚未评测。

1.3 技术挑战

将 JHQ 移植到 GPU 会遇到三个技术挑战。

第一，压缩编码减少了存储，但扫描编码仍可能代价较高。每个八位主码索引一张 256 项的距离查找表（lookup table，LUT）；这些表需要构造，且可能无法放入快速的片上内存。由于 JHQ 通过组合各坐标的量化等级形成编码，这张表可以在实数运算下精确拆成两张 16 项表，分别对应一个字节的前、后四位。挑战在于：更小的表能否抵消额外查表和加法的开销。

第二，小批量可能只有少量查询，不足以充分利用 GPU。将每条查询的候选分给多个线程块可以增加并行工作，但如果各块仅保留相同份额，最好的候选又集中在一个块中，就可能丢失全局优胜者。一种办法是在归并前，让每组各自保留完整的候选短名单，代价是增加

显存和归并开销。挑战在于保留所有可能胜出的候选，同时让这些开销低于并行带来的收益。本文研究共享截断阈值和随批量变化的调度，以降低这些开销。

第三，达到目标召回率所需的精排工作量随工作负载变化。候选过少可能丢失真实近邻，过多则可能花时间读取和计算残差编码，却不改善答案。目标是在不使用真值标签的情况下选择可复用预算。较大预算搜索可以提供参考，但它自身也可能漏掉真实近邻。因此，既要在独立测试查询上核查召回率，也要测量校准和后续搜索的成本。

1.4 研究问题

本文的总问题是：如何通过面向表示结构的执行和工作负载级精排校准，将 JHQ 实现为高效、常驻显存的 GPU ANN 系统。面向表示结构的执行利用编码结构设计数据布局与评分；工作负载级校准从无标签查询中选择一个可复用的精排预算。构建与容量研究提供系统实现基础，跨系统比较则检验其实用价值。由此提出以下四个研究问题。

RQ1：GPU 索引构建与容量。 能否在一张 GPU 上构建并常驻完整的 JHQ 索引？其构建时间、峰值显存和常驻容量分别是多少（第7.7节）?

RQ2：面向表示结构的执行。 如何利用 JHQ 的编码结构设计 GPU 查表、内存访问和候选选择，在遵循各阶段规定的排序与并列规则的同时提高吞吐（第7.3节、第7.4节与第7.6节）?

RQ3：工作负载级精排校准。 能否不使用真值标签，为每个工作负载校准一次精排预算，使独立测试查询上的召回率接近较大预算搜索的召回率，并收回校准成本（第7.5节）?

RQ4：跨系统搜索性能。 按主机到主机边界计时，在什么召回率目标和批量大小下，JHQ-GPU 优于已测试的 GPU ANN 基线（第7.2节）?

第 3 章将这些问题转化为受控比较，分别考察构建了什么、执行是否正确、工作负载需要多少计算，以及系统在哪些区间具有实际价值。

1.5 研究贡献

本文的贡献是 GPU 索引构建流水线、围绕 JHQ 编码设计的搜索执行，以及工作负载级精排校准。表 1.1 将这些新增工作与 JHQ 量化方法及已有 GPU 算法区分开来；后两者属于前人工作。

表 1.1. 技术贡献与支持证据。

	继承基础	项目贡献	证据
C1	JHQ 的变换与层级编码	GPU 模型拟合 / 编码、分批构建及按列表顺序排列的编码	17.78M 向量搜索，常驻 20.44 GiB；拟合与编码为 10.54–10.65 s；采样构建峰值 22.58 GiB
C2	JHQ 编码结构与已有 GPU 算法	双表查找、高效编码读取、使用共享截断阈值的并行选择及协作评分	双表查找在 22/24 配置领先；batch 8 调度吞吐为单 block 的 2.727×；分组选择测试与对相同分数的 CPU 排序一致
C3	候选扩展与输出比较	无标签预算规则、预算搜索条件，以及在独立查询上的质量 / 成本评估	使用完整校准集时，12/12 配置相对最佳已测预算的召回损失最多为 0.0006；四个配置直接测量复用时间

个人贡献。 导师 Mengxuan Zhang 博士提出了将 JHQ 移植到 GPU 的研究方向，并通过定期讨论提供指导。我完成了表 1.1 所概括的工作，包括实验设计、基线比较与分析，并决定采用哪些方案和结论。AI 工具提供了实质性辅助，具体见研究与 AI 辅助说明。

第 2 章背景与相关工作

本章介绍全文使用的搜索目标、压缩表示与 GPU 执行概念，再将 JHQ-GPU 与图索引、量化搜索和自适应精排方法进行对照。

2.1 距离与召回率

对第 1 章介绍的搜索任务，平方欧氏距离为

$$D_{\mathrm{raw}}(q, x) = \|q - x\|_2^2. \tag{2.1}$$

评测对每条查询使用十个存档真值（ground truth，GT）近邻 ID。等距结果也沿用该固定 GT 前缀的选择（式 (3.1)）。

2.2 IVF、PQ、ADC 与精排

倒排文件（IVF）索引将数据库划分为多个列表，通常按向量最近的质心进行分配。查询搜索其中 n_{probe} 个列表。若 $\mathcal{L}(q)$ 表示选中列表的编号集合，则可参与评分的向量为

$$U(q) = \bigcup_{\ell \in \mathcal{L}(q)} \mathrm{list}_\ell, \qquad n_c = |U(q)|.$$

位于 $U(q)$ 之外的真实近邻，在向量评分开始之前就已被排除。

乘积量化（product quantisation，PQ）将向量坐标划为 M 个子空间，为每个子空间学习码本，从而压缩向量 [17]。在子空间 m 中，编码 z_m 指向代表向量 $\mathcal{C}_m[z_m]$；这些代表向量共同重建 $\widehat{x}$。非对称距离计算（asymmetric distance computation，ADC）保留未量化的查询，计算

$$\|q - \widehat{x}\|^2 = \sum_{m=0}^{M-1} \|q_m - \mathcal{C}_m[z_m]\|^2. \tag{2.2}$$

其中 q_m 是查询的子向量。每条查询先计算到各码字的距离并制成表，随后扫描数据库编码时使用查表。PQ 分离了不同子空间，但任意学习得到的码本未必能进一步分离同一子空间内部的坐标。JHQ 的主码本具有这一额外结构。

精排对初步评分选出的候选短名单投入更多计算 [10, 18]。令 α 为精排倍率，$c = \max\{k, \lceil \alpha k \rceil\}$ 为请求的短名单大小。在 $|U(q)| \geq k$ 时，以 $C_\alpha(q)$ 表示至多 c 个保留候选，以 $\mathcal{R}_\alpha(q)$ 表示最终返回的 k 个向量，则

$$\mathcal{R}_\alpha(q) \subseteq C_\alpha(q) \subseteq U(q) \subseteq \mathcal{X}. \tag{2.3}$$

若 $\mathcal{T}_k$ 为该查询的 GT 集合，由包含关系可得

$$|\mathcal{R}_\alpha \cap \mathcal{T}_k| \leq |C_\alpha \cap \mathcal{T}_k| \leq |U \cap \mathcal{T}_k|.$$

增加 probe 数会扩大路由候选池；增加 α 会扩大请求的短名单。要区分路由、粗筛或重构导致的召回损失，需要在保持其他阶段固定的条件下逐项比较。下一节定义 JHQ 的两级评分。

2.3 JHQ 表示及其假设

2.3.1 旋转与层级编码

JHQ 首先对数据库向量和查询应用方形正交变换 $\Pi \in \mathbb{R}^{d\times d}$ [16]。记 $y = \Pi x$、$q' = \Pi q$，则在实数运算下，

$$\begin{aligned}\Pi^\top \Pi = I \Longrightarrow \|\Pi(q-x)\|^2 \\ = (q-x)^\top \Pi^\top \Pi (q-x) = \|q-x\|^2.\end{aligned} \tag{2.4}$$

因此，量化之前的旋转保持维度和距离。有限精度矩阵乘法在第 4 章单独评估。

主码给出 y 的粗略重构 $\widehat{p}(x)$。在评测配置中，每八个坐标组成的子空间使用一个 byte：每个坐标取 $-a$ 或 $+a$ 两档，因此该 byte 记录八个符号位。一般而言，M 个子空间各有 $D_s = d/M$ 维，用 B 位从 $L = 2^{B/D_s}$ 个标量等级的 $K = 2^B$ 种组合中选择。对 L 为整数的支持参数，JHQ 使用 Cartesian 积

$$a_i = \sigma\Phi^{-1}\left(\frac{i-1/2}{L}\right), \quad i = 1,\dots,L, \qquad \mathcal{C}_m = \{a_1,\dots,a_L\}^{D_s}. \tag{2.5}$$

其中 Φ 为标准正态累积分布函数，σ 为根据向量能量估计的尺度。当 $B = D_s = 8$ 时，$L = 2$、$\mathcal{C}_m = \{-a, +a\}^8$，且 $a \simeq 0.67449\sigma$。单位归一化输入使尺度接近 $1/\sqrt{d}$。变换与层级表示均继承自 JHQ。

残差编码用 $\widehat{r}$ 近似剩余误差 $r = y - \widehat{p}$，形成重构 $\widehat{y} = \widehat{p} + \widehat{r}$。评测中的残差量化器学习每个子空间内各坐标共享的标量等级，每个坐标使用一个 byte。因此，每个向量在元数据之外存储 $d/8$ 个主码 byte 和 d 个残差 byte。降低精排预算减少残差读取，已存储的编码保持不变。

该残差相对于主码重构计算。相比之下，使用残差编码的 IVF-PQ 减去向量 x 所属列表的质心 $\mu_{\ell(x)}$：

$$r_{\text{IVF-PQ}} = x - \mu_{\ell(x)}, \qquad r_{\text{JHQ}} = \Pi x - \widehat{p}(x).$$

JHQ 使用 IVF 路由，重构向量时不加回 IVF 质心。

2.3.2 主码与精排评分

主码粗筛比较 q' 与粗略重构：

$$D_{\text{P}} = \|q' - \widehat{p}\|^2.$$

只有短名单中的候选才使用主码和残差码重新评分。实现中的精排分数为

$$D_{\text{H}} = \|q' - \widehat{p} - \widehat{r}\|^2 + \|q'\|^2. \tag{2.6}$$

附加项 $\|q'\|^2$ 对同一查询的所有候选都是常数，因此该分数按重构向量的平方距离排序；这一顺序未必与 GT 使用的原始向量顺序相同。分数展开见式 (5.20)。

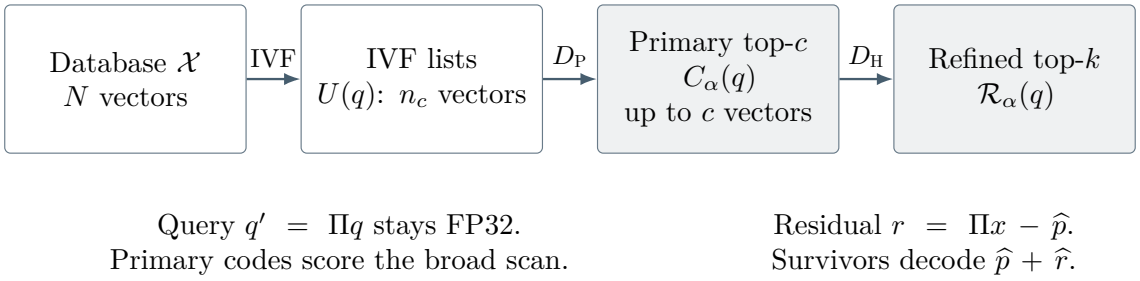


图 2.1. JHQ 的候选与评分级联。IVF 选择列表，主码评分形成短名单，残差码精排该短名单，再执行最终 top-k 选择。

若路由得到 n_c 个向量、主码有 M 个子空间、保留至多 c 个候选，评分工作量为

$$W_{\text{primary}} = O(n_c M) \text{ lookups}, \qquad W_{\text{refine}} = O(cd) \text{ coordinates}.$$

内存访问和候选选择也会增加搜索时间。

2.3.3 建模假设

随机旋转可以分散坐标能量，为上述高斯启发的等级提供动机。距离保持并不意味着坐标独立且服从零均值高斯分布：固定变换将数据库均值 μ 映为 $\Pi\mu$，且 $\|\Pi\mu\| = \|\mu\|$。第 8 章将中心化作为另一种编码选择进行考察。

主码等级取自高斯分位区间的中点，而非使重构均方误差（mean-squared error，MSE）最小的等级。对 $X \sim \mathcal{N}(0, \sigma^2)$ 和零判定阈值，后者为

$$\begin{aligned} a_{\text{MSE}} &= \arg\min_a \mathbb{E}[(X-a)^2 \mid X > 0] \\ &= \mathbb{E}[X \mid X > 0] = \sigma\sqrt{2/\pi} \simeq 0.79788\sigma. \end{aligned}$$

这些数值服务于不同目标。对固定的符号码，任意正的公共主码尺度保持主码排序（式 (4.3)），但残差重构仍取决于尺度。因此，这些代数评分性质不能证明量化最优或原始向量上的召回率。

2.4 GPU 执行概念

GPU 工作组织为线程块；块内线程可通过共享内存协作。warp 是一起调度的 32 个线程。寄存器、共享内存以及常驻线程块数量的限制，共同约束可并发执行的工作量 [25]。

搜索可沿查询、候选和向量坐标三个方向分配工作。

候选选择从扫描池中保留最小分数。Faiss 使用 warp 级选择 [18]；其他 GPU top-k 方法将选择工作分布到更大的数组上 [39]。第 5 章解释 JHQ-GPU 使用的选择与调度方法。

2.5 基于图的 ANN 索引

图索引连接相近向量，沿这些边寻找有希望的候选，而不是扫描选中列表中的全部向量。HNSW 使用分层邻近图进行由粗到细的导航 [21]。在 GPU 上，难点是在遍历过程中提供足够的并行工作。SONG 研究并行图搜索 [40]；CAGRA 面向 GPU 设计图构建与搜索 [28]。本文以 CAGRA 作为常驻图索引基线，测试 FP32 和量化向量两种配置。

其他设计通过跨越存储边界扩展容量。DiskANN 将图搜索与 SSD 结合，用于超过主存容量的数据库 [34]；BANG 将主机上的图与压缩 GPU 评分结合 [38]。这些容量增益依赖跨存储边界的通信。

2.6 压缩搜索与自适应精排

优化乘积量化（optimised product quantisation，OPQ）学习旋转和子码本，以减少重构误差 [13]。ScaNN 的各向异性量化则面向内积近似 [15]。这种区别很重要，因为最小化向量重构误差未必最有利于保持近邻排序。

RaBitQ 用随机二进制编码估计距离 [11]，其多位扩展提供更细致的表示 [12]。GPU IVF-RaBitQ 将压缩距离估计、IVF 路由与融合搜索结合 [33]，作为直接的压缩搜索基线，用来检验 JHQ 额外的残差评分是否值得。

Fast Scan 与 Quicker ADC 使用通过单指令多数据（single-instruction, multiple-data, SIMD）操作访问的小表，加速 CPU 距离计算 [1, 2]。因此，表大小与硬件查表支持对速度至关重要。JHQ-GPU 利用主码本内部的坐标结构拆表；第 5 章分析恒等式与有限精度影响。

Faiss 和 GPU 搜索库 cuVS 提供控制精排候选数的参数 [7, 10]。Faiss 自动调参使用带标签的评估 [9]；学习式提前终止预测何时停止 [19]；DARTH 用召回目标表达搜索工作量 [6]。这些方法为第 6 章的无标签校准提供比较依据。

2.7 与已有方法的关系

表 2.1 将上述方法与 JHQ-GPU 对照。小表方法为利用编码结构提供动机；图索引与二进制编码索引提供不同的速度与容量权衡；自适应计算量方法提供质量控制的比较依据。第 4–6 章展开这些设计选择，第 7 章进行评估。

表 2.1. 相关方法及其引出的 JHQ-GPU 设计问题。

方案	主要权衡	JHQ-GPU 的问题
IVF-PQ / OPQ	学习码本可减少失真，但未必在内部按坐标可分	主码的坐标结构如何简化 GPU 评分?
Fast Scan / Quicker ADC	小表在硬件和精度约束下加速查找	拆表能否抵消额外 GPU 查表?
SONG / CAGRA	图导航减少评分，但增加图存储和依赖访问	压缩扫描在哪些区间能与常驻图搜索竞争?
BANG	主机存储扩大容量，但增加通信	单 GPU 常驻完整压缩索引能实现什么?
IVF-RaBitQ	编码大小与 probe 工作量权衡距离估计质量和速度	残差精排何时值得额外开销?
自适应计算量	质量控制可能需要标签、预测或校准	工作负载一致性能否以可接受的质量和成本选择有效预算?

第 3 章研究设计与测量

成对执行、留出集校准和系统前沿使用不同的对照，本章予以定义。

3.1 实验比较与控制变量

表 3.1 定义比较单元，以及什么结果会挑战各项研究主张。一个配置单元（*cell*）固定数据集、索引配置和工作负载设置；同一单元内的实验臂对应不同实现或预算。对成对执行测量，除被明确指定的处理外，索引状态、查询、probe 数、精排预算及评分表示均保持固定。记录的成对比值为 $Q_{\text{treatment}}/Q_{\text{control}}$。对所说明的 J 个单元等权汇总，得到

$$G = \exp\left(\frac{1}{J}\sum_{j=1}^{J}\log\frac{Q_{\text{treatment},j}}{Q_{\text{control},j}}\right).$$

表 3.1. 将研究问题转化为可测试的比较。容量观察与成对执行干预使用不同的对照。

问题	干预 / 比较	固定或统计的量	指标与反证
RQ1：构建与容量	GPU 构建；缓存与冷生命周期；压缩与原始存储	N, d、表示、路由及构建计时边界	Train / add 时间与常驻字节；构建失败或模型—分配差距界定可行性
RQ2：执行	LUT 分组、word 访问、精排 lanes、分组扫描、共享阈值与 probe 选择	成对状态、负载、分数及参考输出含义	Wall QPS、阶段时间和输出检查；减速或非并列不一致会挑战机制
RQ3：预算	抽样 / 全池规则，对照固定 64、100、200、枚举及 GT oracle	同一索引与路由；不相交的校准和留出查询位置	留出召回损失、所选臂 QPS 与校准成本；越阈挑战可靠性
RQ4：竞争区间	独立调优的 JHQ、CAGRA、IVF-PQ 和 IVF-RaBitQ	导出向量、GT、k、设备及主机到主机边界；显存单独报告	匹配召回 QPS 与覆盖；其他召回目标上的更强对手挑战广泛优越性

消融估计速度效应，不直接测量缓存 / 占用率变化。跨系统比较另行进行（第7.6节）。

3.2 数据集、查询、真实近邻与划分

表 3.2 记录本文使用的六个数据集。主要跨系统比较（E1）使用前四个；预算、LUT 和规模测试覆盖全部六个。Vogue 是服装图像嵌入；arXiv 是摘要嵌入；两个 OpenAI 数据集来自 Qdrant 的 DBpedia 导出 [5, 31, 32, 35]。BGE 使用意大利语 Wikipedia 配置，Stella 使用 TREC24 BioGen 语料导出 [36, 37]。规模描述的是准备后实际测量的导出版本，而非上游当前数据卡。

表 3.2. 数据与 JHQ 训练配置。n_q 为实际查询数，$n_t = 39n_{\text{list}}$ 为请求的粗量化训练样本数。E1 使用前四行。所有配置使用平方 L2 距离及 $k = 10$。

数据集	N	d	n_q	n_{list}	n_t
Vogue	932,328	768	1,000	4,096	159,744
arXiv	2,253,000	768	1,000	8,192	319,488
OpenAI-1536	999,000	1,536	1,000	8,192	319,488
OpenAI-3072	999,000	3,072	1,000	8,192	319,488
BGE-M3	10,091,524	1,024	1,000	32,768	1,277,952
Stella	17,776,615	1,024	999	32,768	1,277,952

OpenAI / BGE / Stella 准备阶段删除非有限行，并使用

$$x \leftarrow x/\|x\| \quad (\|x\| > 10^{-12}).$$

种子 42 用于无放回抽样。Stella 使用 999 个语料内查询，替代原有 65 个查询的基准；没有相关性标签。OpenAI 检查覆盖全部查询和 10,299 个基础行，支持行序对齐，但未按原始 ID 连接（第7.2节与第7.2.3节）。

各方法共享导出的查询 / GT 文件。对 FP32 向量进行分块穷举平方 L2 搜索，维护 GT top-k，包括并列：

$$\text{Recall@}k = \frac{1}{|\mathcal{Q}|k} \sum_{q \in \mathcal{Q}} |\mathcal{R}_k(q) \cap \mathcal{T}_k(q)|. \tag{3.1}$$

这里 $\mathcal{T}_k$ 是存储 GT 的前缀，$\mathcal{R}_k$ 是返回 ID 集合。GT 宽度在 Vogue / arXiv 上为 100，其余为 20；只计前十项。重复 ID 不获得额外命中。越界、重复和重复调用检查分别进行。

JHQ 使用 $M = d/8$、$B = B_r = 8$、八次粗量化迭代和 100,000 行残差训练数据。参数扫描重新载入 / 编码；冷构建重新拟合。训练使用数据库行。

$$\mathcal{Q}_{\text{cal}} = \{q_i : i \bmod 2 = 0\}, \qquad \mathcal{Q}_{\text{hold}} = \{q_i : i \bmod 2 = 1\},$$

$$(|\mathcal{Q}_{\text{cal}}|, |\mathcal{Q}_{\text{hold}}|) = \begin{cases} (500, 499), & \text{Stella}, \\ (500, 500), & \text{otherwise}. \end{cases}$$

每个单元对大小为 32、64、128 的带种子子样本各抽取 64 次；FULL 使用全部 500 个校准查询。标签用于评估 oracle 与留出集；一致性只使用输出（第3.5.2节与第6.2节）。

3.3 计时与显存测量

测量区间为主机搜索调用：

$$t_{\text{call}} = t_{\text{return}} - t_{\text{entry}}, \quad \begin{cases} \text{entry: FP32 queries in host memory,} \\ \text{return: host IDs/scores after stream completion.} \end{cases}$$

暂存、传输、查询计算和 int8 查询量化均计时；构建 / 拟合不计入（图 3.1）。句柄和输出缓冲复用。JHQ 使用 pageable 内存 / C++，CAGRA / PQ 使用 pinned 内存 / Python，RaBitQ 使用 pinned 内存 / C++；这些成本会影响小幅领先。

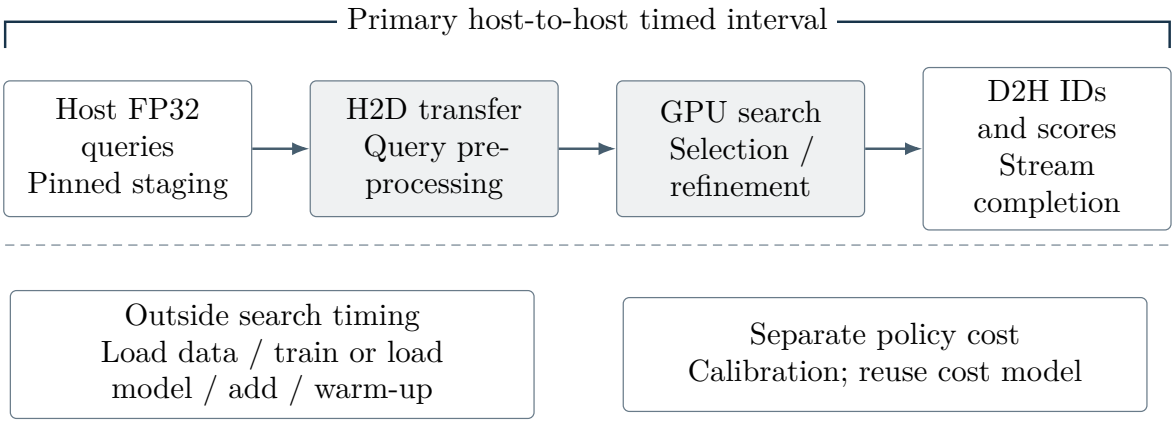


图 3.1. 计时边界。主机区间包含传输和查询专属计算。设备阶段剖析解释其中部分时间；独立剖析和同步可能改变执行，因此阶段时间之和不能替代端到端延迟。

时间单位为秒：

$$\bar{t}_r = \frac{1}{m_r} \sum_{i=1}^{m_r} t_{ri}, \qquad Q_r = \frac{b}{\bar{t}_r}, \qquad Q = \text{median}_r\, Q_r. \tag{3.2}$$

E1：batch 1,000，五轮，通常每轮 24 次调用。JHQ 每个变体 / 轮次预热一次，基线起初预热三次；较慢的基线测点调用更少。即时编译（just-in-time，JIT）和 CUDA Graph 捕获不计入，调用时间分位数不代表并发请求。

有效的慢调用保留。以示意调用时间 $(1, 1, 10)$ ms 为例，

$$\bar{t} = 4\,\text{ms} \neq \text{median}(t) = 1\,\text{ms}, \qquad b/\bar{t} \neq b/\,\text{median}(t).$$

共享索引 / 查询池不能提供独立构建样本。

RTX 5090 / 32,607 MiB；驱动 595.71.05；CUDA 13.0；cuVS 26.08.01。OpenMP 使用 25 个 CPU 中的 16 个；E1 记录功率 575 W。频率 / 负载可能变化。JHQ 提供 compute_80 PTX 中间 GPU 代码，由驱动编译 [27]；库构建与设备剖析另行处理。

常驻总量包含模型、工作区和上下文。CAGRA 索引增量与 JHQ 扣除上下文后的总量口径不同（第4.4节与第7.7节）。内存池 / 临时区妨碍等峰值比较。构建峰值每 20 ms 采样；train / add 不包含数据载入和主机准备。

补充实验会话。 各 campaign 独立（第3.5.2节）；缓存状态 / 精度变化见 第4.2节与第7.2.3节。与历史基线的比值仍是跨会话比较。

3.4 基线与前沿构造

CAGRA fp32 / int8、IVF-PQ 和 IVF-RaBitQ [17, 28, 33] 使用各自的网格（表 3.3）；FP32 brute force 为穷举搜索。存储和重建精度可能不同。因此，主前沿衡量的是各自网格内达到质量目标时可实现的速度，并附单独的显存统计。

表 3.3. E1 的主要调参维度。精确逐点参数与覆盖扩展保留在原始记录中。网格有限，且不同方法的网格不同。

家族	记录的搜索维度
JHQ	Probes 与 α；初始 probes 为 64/128/256，预算为工作负载预算的 0.5/1/2 倍；覆盖扩展至 probe 1,024 和更大 / 更小预算
CAGRA fp32/int8	图度数 32/64；搜索宽度 1/2/4/8；中间 top-k 容量 32–512
IVF-PQ	八位子码；$d = 768$ 时编码 192/384/768 bytes，更高维为 384/768/1,536；fp16/fp32 查表；probes 16–512
IVF-RaBitQ	每维 4/8/9 位、支持的搜索模式、probes 16–1,024；额外 4,096 列表 OpenAI 索引
GPU 暴力搜索	FP32 平方 L2 穷举，共同主机边界

支配关系与实测前沿定义为

$$a \succ b \iff R_a \geq R_b,\ Q_a \geq Q_b,\ (R_a, Q_a) \neq (R_b, Q_b).$$

$$Q_{\text{meas}}(r) = \max\{Q_i : R_i \geq r\}. \tag{3.3}$$

没有满足要求的实测点时，结果未定义。对相邻前沿点，

$$\log Q_{\text{int}}(r) = (1-\lambda)\log Q_a + \lambda \log Q_b, \qquad \lambda = \frac{r - R_a}{R_b - R_a}. \tag{3.4}$$

在 5×10^{-4} 内，解析器吸附到测点；否则只做插值，不外推。小批量比较中满足阈值的端点仍可入选。接近持平时同时报告两个估计器；显存未做匹配。

失败 / 无效输出被排除；仅有重复 ID 标记不会导致排除。隔离重跑替代受污染的构建；跳过的 Stella batch-1,000 运行不产生测点（第3.5.2节）。Int8 仍纳入比较（图 7.1）；ADC / 海报图使用各自范围。

3.5 实验记录与可复现性

实验代号区分不同的测量家族。E1 是 batch-1,000 的主要前沿扫描；D 将采用 TensorFloat-32（TF32）和 FP32 算术的 JHQ 与存档基线比较，T1 则在同一会话重测基线。T4 对 GPU 分数进行 CPU 排序；T5 测试小 batch。A1 / A2 / A3 分别表示重复建库、共享阈值指令和数据集对齐；B 记录预算决策；C1 审计表示。

3.5.1 来源与可用记录

项目资料库保留版本、配置、计时、输出，以及选定的 T1、T4/T5 和 A2 输入。成对运行共享索引状态。完整向量、训练状态和历史可执行文件 / 哈希并非全部可用。T4 候选转储已删除，保留的汇总无法支持独立重新排序。抽样与泛化限制见 第6.2节、第6.3节与第8.3节。

表 3.4. 组件测试使用的实现家族。各实验有自己的对照；版本号不表示连续累积的整系统加速。

版本	机制 / 实验作用
v61	Word 载入与 lane 精排对照
v62 / v63	分组完整键选择 / 共享阈值
v65 / v69	Batch / 偏斜调度及 radix probe 选择
v69 / v71 / v73	预算校准 / LUT 扫描 / 严格分数导出
v69 / v72	OpenAI：同一训练缓存上的 TF32 与 FP32 add / 查询算术

3.5.2 计时与校准约定

预热使 E1 中 274 个测点的调用数减少；首次转换 / 验证不同于稳态计时。369 个重复 ID 标记包含并列重排；401 的口径还计入跨协议差异。仅凭标记不排除测点，JHQ 没有同样的标记。

实验显式设置以 200 开始的网格。固定实验臂预热一次、计时三次。校准计时包含完整搜索与 $O(Sk^2)$ 主机比较，不含查询收集 / 扫描调度校准，且绕过图捕获。预热遍历全部查询，但不将输出 / GT 传入一致性判定。调用方使用 `set_alpha(A[index])` 应用返回的索引；所选实验臂速率复用固定臂计时（第6.5节）。对排序后的 64 个损失，记录分位数为

$$L_{95} = L^{\text{sorted}}_{\lfloor 0.95(64-1) \rfloor} = L^{\text{sorted}}_{59} \qquad \text{(zero-based)}.$$

除非另有说明，后续会话使用独立状态 / 边界。A2 佐证 PTX 约束（第5.5节）；T4 分数导出的范围见 第7.4.1节。T5 包含空调用 / int8 查询量化，也存在 API / 预热差异（第7.2.6节）。

第 4 章GPU 索引设计与构建

训练 / 编码生成有界内存、按列表排序的 GPU 数组 [16]；搜索与成本见 第5章与第7.7节。

4.1 构建与搜索流程

train 拟合模型，add 将行编码并按列表排序，search 使用完成的索引。缓存命中跳过拟合，但不跳过编码，因此冷构建与缓存搜索的成本不同。

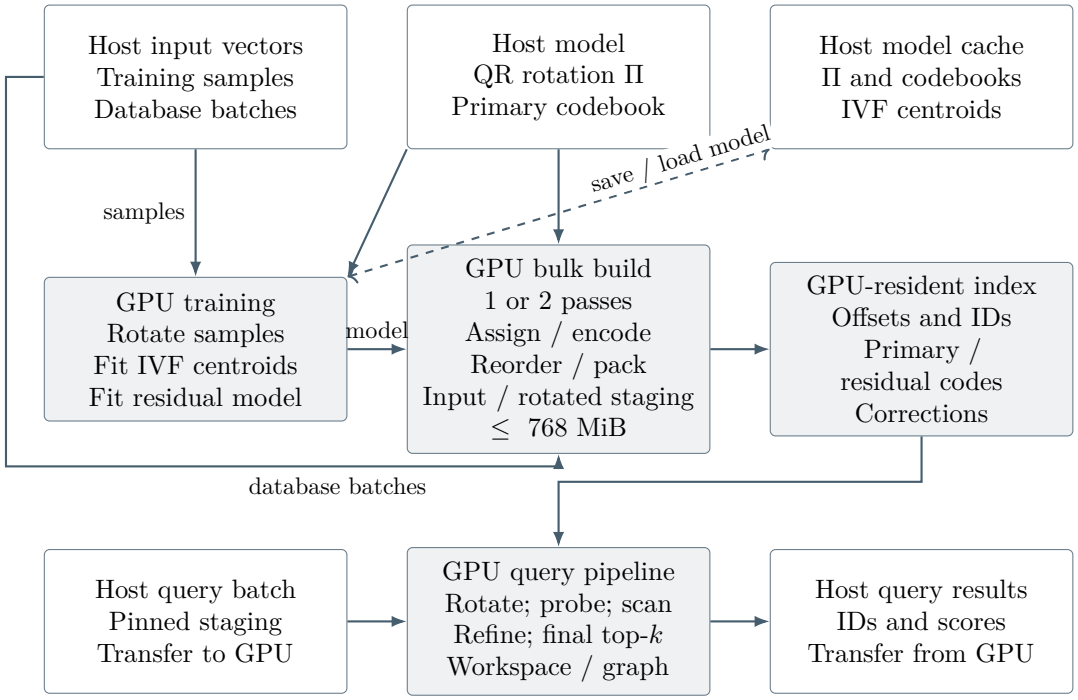


图 4.1. 实现中的构建与查询边界。主机工作包括抽样、正交矩阵构造和解析主码本生成；评测配置在 GPU 上拟合 IVF 与残差模型。缓存保存训练状态，完整数据库表示则位于设备上。虚线表示模型复用。暂存上限不包含其他临时空间。

主机准备样本、旋转和主码本；GPU 执行旋转，拟合 IVF / 残差模型，并将有界 batch 编码为列表顺序。

模型保存旋转、码本和质心；数据库状态保存 offsets、ID、编码和校正项。CUDA Graph 重放已记录的 GPU 操作，并引用复用的查询缓冲。

一次成功的 `add` 固定物理位置。后续 add 被拒绝，更新需要重建。可变 / 事务性发布尚未实现。

主机 ID / 分数及调用计时遵循 式 (5.20) 与第3.3节。

4.2 训练、旋转与码本生成

4.2.1 变换与抽样

对带种子的高斯矩阵在主机上做 QR 分解，得到方形 Π：

$$Y_t = X_t \Pi^{\mathsf{T}}, \qquad y = \Pi x, \qquad \|\Pi(q-x)\|_2^2 = \|q-x\|_2^2 \quad \text{when } \Pi^{\mathsf{T}}\Pi = I. \tag{4.1}$$

几何契约 C0 检查有限精度 QR / GEMM 误差；维度仍为 d（第4.6节）。

主码 / IVF 训练按步长选取 $n_t = 39n_{\text{list}}$：

$$n_t = \begin{cases} 159{,}744, & \text{Vogue,} \\ 319{,}488, & \text{arXiv/OpenAI,} \\ 1{,}277{,}952, & \text{BGE/Stella.} \end{cases}$$

这是 Faiss 的警告阈值，不是质量保证 [8]。残差拟合使用 100,000 行。

较早的十二单元 Vogue / OpenAI-3072 扫描（$B_r = 4/8$，probe 为 32/128/256）固定主码 / IVF 模型。在 25 次 Lloyd 迭代下，以 100K 行替代全部行最多损失 0.0018 召回率（十一个下降、一个持平）。中间样本量的结果并非单调；该结果只在这一协议内支持有界训练。

4.2.2 解析主码本

样本尺度和 Cartesian 等级为

$$\sigma^2 = \frac{1}{n_t d} \sum_{i=0}^{n_t-1} \|x_i\|_2^2, \qquad D_s = d/M, \qquad K = 2^B. \tag{4.2}$$

$$B = D_s = 8 \Longrightarrow \mathcal{C}_m = \{-a, +a\}^8, \quad a = 0.674\sigma.$$

八个符号可放入一个 byte，支持 $O(d)$ 编码与拆分 LUT [16]（第5.2节）。任意 Lloyd 更新可能破坏 Cartesian 结构。

对固定符号向量 $s \in \{-1, +1\}^d$，

$$\|q' - as\|^2 = \|q'\|^2 + da^2 - 2a\langle q', s\rangle. \tag{4.3}$$

$$a > 0 \Longrightarrow \arg\min_s \|q' - as\|^2 = \arg\max_s \langle q', s\rangle.$$

这一排序需要 $a > 0$，而不需要数据为高斯分布。残差依赖 a；零尺度和 FP32 并列另行处理。

4.2.3 粗分配中的筛选与精确计算

IVF 用间隔选取的旋转行初始化，并以 $s_\ell = \|\mu_\ell\|^2 - 2\langle y, \mu_\ell\rangle$ 运行八次迭代。支持的维度使用 int8 筛选，在 FP32 中处理不确定情况。

引理 4.1 (具有认证分数区间的分配)**.**
设 $\widetilde{y}, \widetilde{\mu}_\ell$ 为反量化近似，$e = y - \widetilde{y}$，$f_\ell = \mu_\ell - \widetilde{\mu}_\ell$。保持质心范数精确时，近似分数 $\widetilde{s}_\ell = \|\mu_\ell\|^2 - 2\langle \widetilde{y}, \widetilde{\mu}_\ell\rangle$ 的实数算术误差由下式界定：

$$E_\ell = 2\big(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|\big). \tag{4.4}$$

给定包含每个目标分数的有效区间 $[L_\ell, U_\ell]$，仅丢弃 $L_\ell > \min_j U_j$，并对其余分数精确求解，即可保留最小值及规定的并列规则。

证明.
使用 $e = y - \widetilde{y}$ 和 $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$，

$$\langle y, \mu_\ell\rangle - \langle \widetilde{y}, \widetilde{\mu}_\ell\rangle = \langle y, f_\ell\rangle + \langle e, \widetilde{\mu}_\ell\rangle,$$
$$|s_\ell - \widetilde{s}_\ell| \le 2\big(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|\big) = E_\ell.$$

对 $j^* \in \arg\min_j U_j$，

$$L_\ell > U_{j^*} \Longrightarrow s_\ell \ge L_\ell > U_{j^*} \ge s_{j^*}.$$

严格不等号排除 ℓ；等号情况仍可入选。 □

若无累加误差界，FP32 裕量不构成逐位认证。FP16 的尾部分配差异分别为 Vogue 的 0/14,838、BGE 的 6/31,748 和 Stella 的 3/16,359。冷构建日志未标明精度。

4.2.4 残差拟合与迭代预算

残差仍为 $r = \Pi x - \widehat{p}(x)$。每个子空间通过 GPU 排序 / Lloyd 拟合 $K_r = 2^{B_r}$ 个有序标量等级，在坐标间共享，最多更新 2,000 次。

历史 100K 训练在 25→2,000 次更新时增加 0.0063–0.0076 召回率。尾部 bin / BGE / Stella 的变化仍在继续：达到上限不代表收敛，这不违背精确 Lloyd 的下降性质。20,000 次更新时，历史 Stella 全量 / 100K 召回率为 0.9921/0.9920。

4.2.5 构建工作量与观测到的不均衡

$$W_{\mathrm{QR}} = O(d^3), \qquad W_{\mathrm{rotate}} = O(n_t d^2).$$

直接粗分配每次迭代成本为 $O(n_t n_{\mathrm{list}} d)$。

$$W_{\mathrm{primary}} = O(MKD_s), \qquad W_{\text{residual extraction}} = O(n_r d),$$

随后进行排序与标量更新。它们描述工作量，而非预测延迟。同步阶段诊断表明，BGE / Stella 训练约 83% 的时间在 IVF k-means（约 2.97 s）。该诊断使用一个 add 缓冲并强制同步，不同于 第7.7节 中正常的重叠计时。

列表长度不均衡进入 第5.5节 的扫描规则。本文未单独分离八次 IVF 迭代、间隔种子和有限抽样各自对不均衡的影响。

实测实现使用的算术。 评测实现 v69 存储 FP32 矩阵，但在 $d \geq 1{,}536$ 时启用 cuBLAS TF32 tensor-core 运算。其编译后的 faithful 分支中，空覆盖值落入按维度选择的默认分支。这影响两个 OpenAI 数据集在训练 / add / 查询中的旋转和质心 GEMM；环境变量无法关闭该分支。因此，FP32 存储并不表示完全采用 FP32 乘法。补充 FP32 变体 v72 修正了这一开关。敏感性实验复用同一个 TF32 训练缓存，改变 add / 查询算术，并非以 FP32 重新训练模型（第7.2.3节）。

4.3 IVF 分配、ID 与物理内存布局

4.3.1 从数据库 ID 到列表顺序的物理位置

输入行的标识为 $i \in \{0, \ldots, N-1\}$。粗分配根据计算得到的质心分数，将其变换向量放入某个列表。数学分配目标为

$$a(i) = \operatorname*{arg\,min}_{0 \leq \ell < n_{\mathrm{list}}} \|\Pi x_i - \mu_\ell\|_2^2. \tag{4.5}$$

实现采用选定的数值分配路径。主码和残差编码时不减去 $\mu_{a(i)}$；列表归属独立于这些重建，限制后续搜索空间。

构建器将分配结果与输入 ID 序列一同排序。令 $\rho(p)$ 为排序后物理位置 p 上的输入 ID，o_ℓ 为列表 ℓ 的起点。完成的数组满足

$$0 = o_0 \leq o_1 \leq \cdots \leq o_{n_{\mathrm{list}}} = N, \qquad p \in [o_\ell, o_{\ell+1}) \Longrightarrow a(\rho(p)) = \ell. \tag{4.6}$$

通过 lower-bound 搜索得到 offsets，其中

$$o_\ell = o_{\ell+1} \Longleftrightarrow \text{list } \ell \text{ is empty}.$$

位置只在一个快照内稳定，跨构建未必稳定。所有数组使用同一个 $\rho(p)$。

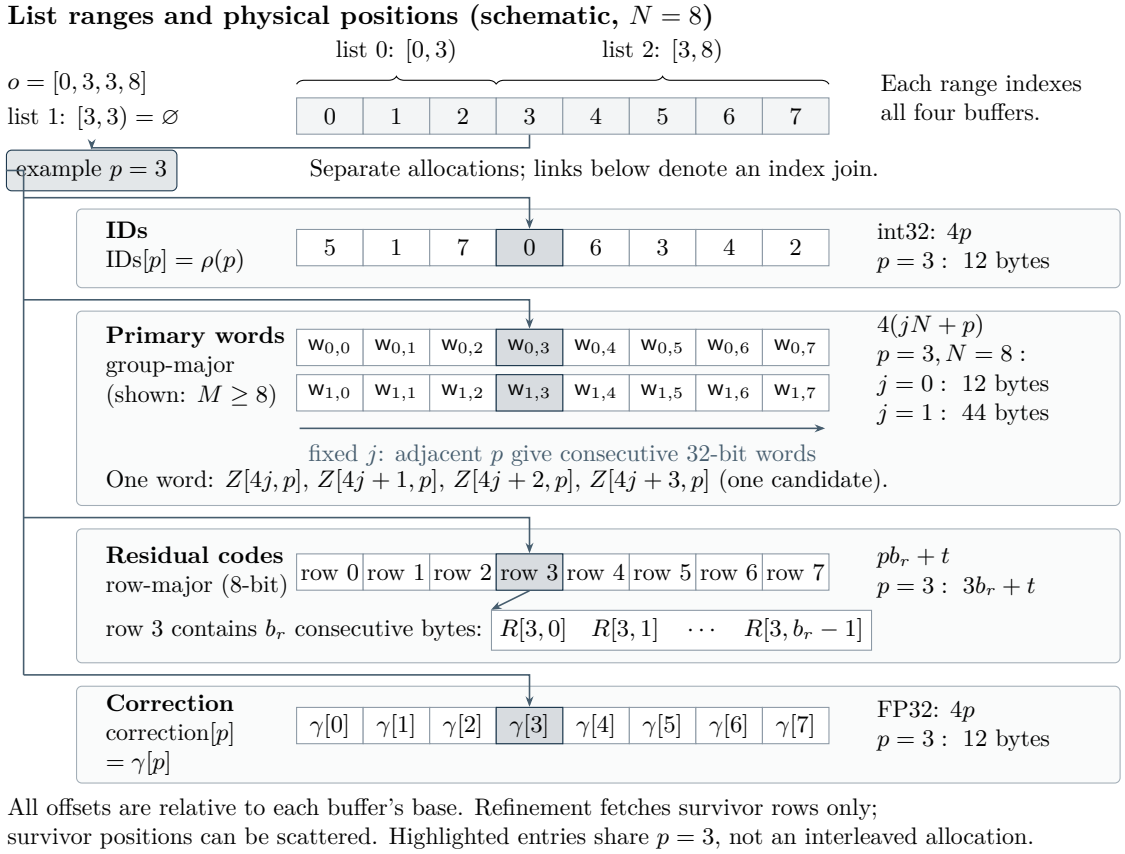


图 4.2. GPU 索引物理布局示意。IVF offsets 划定连续物理位置区间。独立的 ID、word-group-major 主码、row-major 残差和校正数组共享位置 p。byte 偏移相对各自分配起点；图示既不表示交织分配，也不表示实测显存事务。

ID 将 $p \mapsto \rho(p)$ 映射到原行。两遍 add 为 scatter 暂时构造 ρ^{-1}。主码并列使用物理位置，最终并列使用数据库 ID。

4.3.2 精确地址与表示宽度

对主码 byte $Z[m, p]$，

$$\mathrm{offset}_{\mathrm{byte}}(Z[m, p]) = mN + p.$$

最终打包得到

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \mathrm{addr}_{\mathrm{byte}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{4.7}$$

这里 $j = 0, \ldots, M/4 - 1$。word-layout build $\Longrightarrow M \bmod 4 = 0$；其余 M 被拒绝。独立的 byte-layout 编译保留 byte 表示。每个打包数组占 NM bytes，但重打包暂时同时持有新旧副本，成本为 $2NM$。一遍构建器暂时 gather 一个 vector-major 的 $[N, M]$ 主码数组，转置后重打包，并释放过期副本。两遍构建器直接 scatter 到 subspace-major 数组，再进行相同的最终重打包。搜索时的 lane 映射见 第5.3节。

残差码保持 row-major，每向量占 $b_r = \lceil dB_r/8 \rceil$ bytes。八位配置中，行 p 的坐标 t 位于 byte 偏移 $pb_r + t$。四位路径中，它位于 byte $pb_r + \lfloor t/2 \rfloor$，偶数坐标在低 nibble、奇数坐标在高 nibble。每个物理行有四个 byte 的校正项，存储

$$\gamma[p] = 2\langle \widehat{p}(x_{\rho(p)}), \widehat{r}(x_{\rho(p)}) \rangle. \tag{4.8}$$

gather 或 scatter 时，这个值必须与该行编码一同移动。

列表顺序的主码扫描只为保留候选获取残差。若使用原始 FP32 重排序，还需增加 $4Nd$ 个设备 bytes。本文精排压缩重建；第 7 章测量召回率。

4.4 常驻存储、工作区与索引生命周期

4.4.1 存储字节与分配字节

主码以 byte 存储时，

$$S_{\text{rows}} = N\left(M + \left\lceil \frac{dB_r}{8} \right\rceil + 8\right), \qquad S_{\text{offsets}} = 4(n_{\text{list}} + 1). \tag{4.9}$$

ID / 校正项贡献 $8N$ bytes。当 $M = d/8$、$B_r = 8$ 时，

$$b_{\text{code}} = d/8 + d = 9d/8 = 1.125d.$$

编码载荷比 8 位标量量化（SQ8）多 12.5%；不存图并不必然比 int8 更小。

表 4.1. 每向量字节数。JHQ 包含编码、ID 和校正项；SQ8 仅含向量编码；CAGRA int8 模型包含向量编码及 32 个四字节邻居。不含 offsets、模型参数、工作区及其他运行时分配。这些组件不是完整进程总量。

d	主码	残差	ID + 校正	JHQ	SQ8	CAGRA int8	FP32
768	96	768	8	872	768	896	3,072
1,024	128	1,024	8	1,160	1,024	1,152	4,096
1,536	192	1,536	8	1,736	1,536	1,664	6,144
3,072	384	3,072	8	3,464	3,072	3,200	12,288

图度数为 $d_g = 32$ 时，CAGRA 每行使用 $d + 4d_g$ bytes。Stella 的 JHQ 载荷 19.20 GiB 接近 int8 CAGRA 的 19,656 MiB，但统计口径不同。α 改变残差读取量，不改变存储字节。

基本模型存储为

$$S_{\text{model}} = 4d^2 + 4MKD_s + 4MK_r + 4n_{\text{list}}(d+1) \quad \text{bytes.}$$

派生数组另计。工作区保存两个查询矩阵，以及质心分数、LUT、probes 和保留候选。可选残差 LUT 增加 $4b_{\text{cap}}dK_r$ bytes；默认直接评分。即使未使用，128 probes 下的分组容量仍约 93.8 MiB（式 (5.13)）。

输入归调用方所有。独立暂存 / 排序存储为

$$S_{\text{pinned}} = 4n_b a_{\text{in}} d, \qquad S_{\text{order}} = S_{\text{inverse}} = 4N.$$

分配临时区在 gather 前结束生命周期（式 (4.10)与(4.13)）。

4.4.2 缓存复用与失效处理

缓存键不含代码版本 / 全量数据哈希。代码 / 数据变化后需使缓存失效。扫描共享状态，冷构建重新拟合。召回率相近不能证明模型相同。

缓存命中仍需 QR、预热、模型上传和 add。Stella 的 6.67 s add 包含初始工作区，不含扫描校准 / 搜索预热，也不是重启时间。

改变 α 无需重训。工作区在图捕获前扩容；改变 batch 大小、保留数、probes、lanes、groups 或 bounds 会触发重新捕获。校准可能绕过捕获。索引拥有的缓冲复用直至释放。

4.5 构建成本与已测规模

4.5.1 有界输入 batch 与两遍构建

全库原始 / 旋转向量存储为 $4Nd + 4Nd = 8Nd$ bytes。有界暂存避免了这一开销。对 n_b 个流水缓冲、配置的 add 行数 a_{add} 及每数组 byte 预算 S_{stage}，源码暂存量为

$$a_{\text{in}} = \max\left\{1, \min\left(a_{\text{add}}, \left\lfloor \frac{S_{\text{stage}}}{4d} \right\rfloor\right)\right\}, \qquad S_{\text{float}} = 8n_b a_{\text{in}} d. \tag{4.10}$$

$$4d \le S_{\text{stage}}, \quad n_b = 3,\ S_{\text{stage}} = 128\,\text{MiB} \Longrightarrow S_{\text{float}} \le 2n_b S_{\text{stage}} = 768\,\text{MiB}.$$

分配 / 编码器临时区另计。

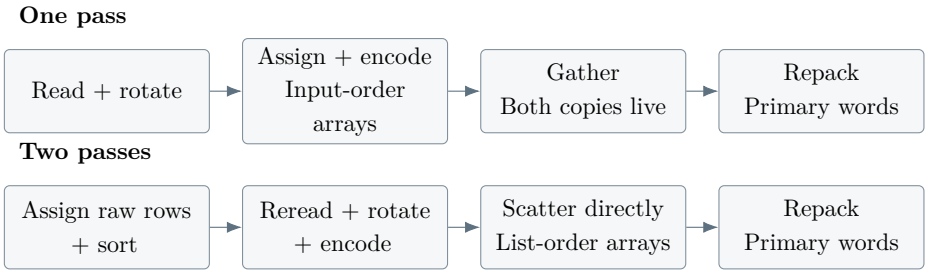


图 4.3. 构建路径。一遍构建短暂同时持有输入顺序和 gather 后的编码；两遍构建重新读取 / 旋转，直接编码为列表顺序。用反向旋转质心对原始向量分配，依赖正交性。

4.5.2 构建各阶段的显存占用

一遍构建的估计为

$$H_{\text{single}} = N(2b_r + 3M + 20) + 2^{30} \quad \text{bytes}. \tag{4.11}$$

其十个存活数组合计

$$3NM + 2Nb_r + 20N = N(3M + 2b_r + 20).$$

1 GiB 余量是启发式，并非进程峰值上界。

两遍构建的最终重打包计入 ID、残差、校正项和两份主码：

$$S_{\text{repack},2} = N(b_r + 2M + 8). \tag{4.12}$$

整个构建峰值还需考虑更早的流水 / 逆置换临时区、模型、排序、上下文和后续工作区分配。

表 4.2. 行数组占用模型（GiB），$d = 1{,}024$、$M = 128$、$B_r = 8$。BGE 使用一遍，Stella 使用两遍。Stella 的一遍列是反事实容量分析。阶段值不含流水、模型、offsets 和运行时成本。

数据集	最终行载荷	一遍 Gather	两遍重打包
BGE	10.90	23.05	12.11
Stella	19.20	40.59	21.32

表 4.2 量化压缩副本的节省。Stella 冷构建日志选择两遍，并对 式 (4.11) 打印 41.6 GiB，而尾部可用量为 30.6 GiB。单独采样的常驻进程总量包含上下文和查询工作区：BGE 为 11.93 GiB，Stella 为 20.44 GiB（第4.4节）。补充的 20 ms 设备采样在 BGE add 开头附近观察到 26.08 GiB 峰值，在 Stella 重打包时观察到 22.58 GiB 峰值。这些是包含上下文的设备采样总量，可能漏掉更短暂的峰值。

分配乘积缓冲容量使用独立配置行数 a_{assign}，由 `JHQ_ASSIGN_BATCH` 或源码默认值选择：至少 8,192 个列表时为 32,768，否则为 8,192。

$$S_{\text{dots}} = 4 n_b n_{\text{list}} a_{\text{assign}} \quad \text{bytes.} \tag{4.13}$$

分配容量独立于当前活跃 / 暂存行数。在 $d = 1536/3072$ 下，两个默认值均分配 3 GiB。BGE / Stella 的 $3 \times 4 = 12$ GiB 分配临时区先于 gather 存活；图 4.4 区分这些生命周期。

$$S_{\text{peak}} = \max_t \sum_{A \text{ live at } t} |A|.$$

Stella 较小的重打包量加模型 / 上下文，对应较低的实测峰值。

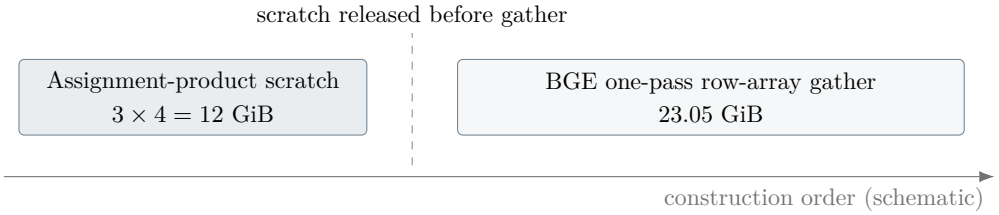


图 4.4. 选定构建分配的生命周期，不是进程总占用。BGE 的 12 GiB 分配缓冲在其 23.05 GiB 行数组 gather 之前释放。这两项不能相加为并发占用。

4.6 索引表示与验证

有符号 32 位的 $n_t d$ 在 2^{31} 溢出，使尺度归零；不检查版本的缓存再次载入了该值。分块 FP64 范数修复了错误；仅观察路由成功无法发现它。

4.6.1 表示一致性检查

试验导出覆盖 Vogue、arXiv 和 OpenAI-3072。

码本通过可分性、标量等级和排序检查。FP64 循环检查每个导出行的 $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$；最大绝对误差为 1.28×10^{-7}–2.77×10^{-7}。抽样 GPU 返回 top-10 分数的最大相对误差为 2.38×10^{-7} 至 4.85×10^{-5}，使用包含查询范数常数的 $|s_{\text{GPU}} - s_{64}|/s_{64}$。这些检查验证导出的表示，而非从原始输入向量重新编码。

试验还在七个工作负载、896 个查询实例上，将提出的残差界与 FP64 重建距离比较，覆盖 72,800,590 个路由候选实例，报告违规数为零。同一批数据也支持 FP64 复合分数恒等式检查。GPU 返回分数检查只覆盖 top-10 槽位，并跳过不在路由映射中的 ID，因此输出集合成员资格尚未完整检查。

规范化命名空间并移除导出方法后，编码器、主码 / 残差训练及构建源码与评测实现相符。这种源码对应覆盖共享路径；后续模型身份与大数据集两遍构建另行检查。

4.6.2 输入编码与数据放置检查

补充导出器审计在 Vogue、arXiv、OpenAI-3072 和 Stella 上使用共享构建路径。offsets 单调，端点为 0 和 N；四个数据集的 ID 均构成所有数据库行的排列。对存储旋转做 FP64 乘积，$\|\Pi\Pi^{\mathsf{T}} - I\|$ 的最大逐元素误差为 3.96–4.34×10^{-7}。抽样范数检查使用 FP64 旋转，而非 GPU GEMM，因此测量的是存储矩阵的几何性质。表 4.3 汇总抽样重编码。

表 4.3. 原始输入的 FP64 重编码审计。Flips 计数坐标决策；末列为不一致项到残差决策边界的最大距离。结构检查覆盖全部行，重编码使用抽样。

数据集	行数	正交误差	主码翻转	残差翻转	最大边界距离
Vogue	200,000	4.11e-07	14	5,409	1.12e-07
arXiv	100,000	4.11e-07	1	2,515	7.31e-08
OpenAI-3072	20,000	3.96e-07	5,745	1,646,806	2.67e-05
Stella	100,000	4.34e-07	16	3,064	1.03e-07

在 768/1,024 维下，重编码差异均距边界不超过 1.12×10^{-7}。OpenAI-3072 有 2.68% 的残差不一致，距边界最高 2.67×10^{-5}，与 TF32 的影响相符。以存储主码为条件，避免重复计算翻转。

按 ID 连接后，三个较小数据库的构建编码完全相同，校正项差不超过 8.94×10^{-8}，Vogue / arXiv / OpenAI 的分配差异为 1/1/40。Stella 仅检查两遍构建。没有导出的质心，因此成员归属尚未验证。

4.6.3 验证覆盖范围与局限

表 4.4. 试验和补充输入审计后的表示约束。检查分别处理几何、算术和行放置。

边界	必要性质	证据 / 剩余检查
C0: 几何	共同变换及有界浮点等距误差	四个数据集检查存储矩阵正交性；区分 TF32 GPU 算术
C1: 码本 / 校正	Cartesian 等级、有序残差码本、校正符合解码编码	三个数据库完成试验检查；全行校正误差见第4.6节
C1: 输入编码	存储码对应预期原始输入行	四个样本的 FP64 原始输入重编码；量化边界和 TF32 差异
C1: 放置	Offsets 覆盖各行；ID 构成排列；所有字段随之移动	全行 offsets / ID 检查；三个按 ID 连接的路径比较；质心归属未验证
执行	有效索引、同步和分数顺序约束	存档查询和 Sanitizer 检查；覆盖有限，详见第7.4节

必要条件为

$$M \mid d, \quad D_s \mid B, \quad M \bmod 4 = 0.$$

要求输入有限、计数有效且哨兵正确。ID 为有符号 32 位；地址乘积使用更宽算术。

T4 验证选择（第7.4.1节）；质心归属、空列表和对抗尾部仍未认证。

第 5 章面向表示的 GPU 搜索

GPU 评分 / 选择使用 JHQ 的层级结构 [16]。第6章与第7章 分别评估执行并设置预算。

5.1 查询流程与排序分数

查询经过旋转、路由、筛选和精排，返回 k 个 ID / 分数。图 5.1 区分运行时的三个选择决策；真实近邻排序是外部评估对象。

必须区分四个量：

$$
\begin{aligned}
s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,}\\
D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,}\\
D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,}\\
D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.}
\end{aligned} \tag{5.1}
$$

μ_i 是变换后的质心，$\widehat{p}, \widehat{r}$ 是解码的主码 / 残差向量。运行时精排分数包含一个仅依赖查询的常数，第5.6节 从实现中的校正项推导该常数。对固定查询，它保持重建距离的顺序，但它和主码分数都不是原始向量距离。

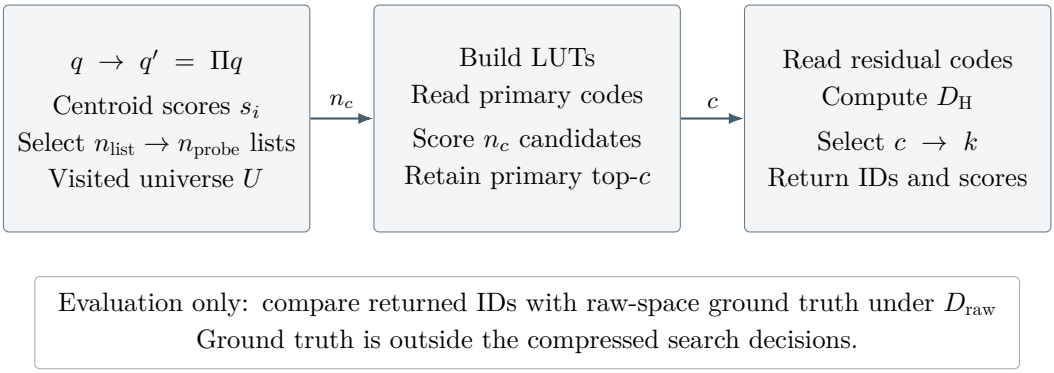


图 5.1. 查询执行及独立的选择边界。路由选择列表，主码筛选选择向量候选，最终选择对精排保留候选排序。原始空间 GT 用于评估，不参与这些搜索决策。

路由定义已访问空间 U，含 n_c 个候选出现实例；主码筛选最多保留 $c = \max\{k, \lceil \alpha k \rceil\}$ 个供残差评分。表 5.1 记录各阶段的输入、读取和成本；实现详见 第5.2至5.7节。

表 5.1 的排序与精确性列将每项主张绑定到对应分数和并列规则。第 7 章分别测试保留集合、有序输出与原始空间召回率。

表 5.1. 查询阶段约束。$c_v = \min(c, n_c)$ 计数有效主码保留候选。精确性针对所说明的计算分数；路由与量化重建决定原始空间近似。

阶段	输入 → 输出	读取数据	主要工作	顺序 / 精确性
变换	$q \to q'$	查询；变换	应用 Π	变换精度见 第7.2.3节
Probe	$n_{\text{list}} \to n_{\text{probe}}$ 个列表	FP32 质心；列表长度	评分；radix；排名计数	(s_i, i)；精确 probe 选择
LUT	$q' \to$ 表	主码本	$MD_s 2^{8/g_{\text{LUT}}}$ 项	实数算术分解；FP32 舍入
主码，$g_{\text{scan}} = 0$	$n_c \to c_v$	打包主码；LUT	$O(n_c M)$ 评分；压缩	按距离精确选择；cutoff 并列任取
主码，$g_{\text{scan}} \geq 1$	$n_c \to c_v$	相同主码表示	评分；压缩；归并	(D_{P}, p)；指定前提下精确
精排	$c_v \to c_v$ 个分数	残差码；校正	$O(c_v d)$	计算的 D_{H}；FP32 结合顺序依赖 lane
最终	$c_v \to k$	精排分数；ID	$O(kc)$ 槽位扫描	$(D_{\text{H}}, \text{ID})$；有 k 个有限有效项时精确

表 5.1 给出阶段约束。E1 / 预算 harness 显式选择 16 个精排 lanes 和扫描策略 -1（在 0/2/4/8 之间调度），而库默认使用标量精排与策略 0。共享阈值已启用，但只在 $g_{\text{scan}} > 1$ 时使用；harness 用 512 个数据库伪查询校准调度，库不会自动执行这一步。组件覆盖配置和实现版本见 表 3.4。

5.2 精确的 Cartesian LUT 分解

主码 ADC 使用 FP32 q' 与存储的重建 [16, 17]：

$$T_m[z] = \|q'_m - \mathcal{C}_m[z]\|_2^2, \qquad D_{\text{P}}(q, x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

因为每个子码本具有 Cartesian 结构，256 项表可以缩小。

对 $K = 256$、$D_s = 8$，解析码本给出

$$\begin{aligned} a &= \sigma \Phi^{-1}(3/4) \simeq 0.67449\sigma, \qquad \mathcal{C}_m = \{-a, +a\}^8, \\ D_{\text{P}}(q, x) &= \|q'\|_2^2 + da^2 - 2a \sum_{j=0}^{d-1} s_j(x) q'_j. \end{aligned} \tag{5.3}$$

其中 Φ 是标准正态 CDF；一个 byte 存八个符号，高位在前。因此

$$a > 0 \quad \Longrightarrow \quad \arg\min_x D_{\text{P}}(q, x) = \arg\max_x \sum_j s_j(x) q'_j.$$

四路 / 八路拆分保持相同的实数算术分数。学习式替代码本需要兼容性检查。

5.2.1 不改变量化器的分解

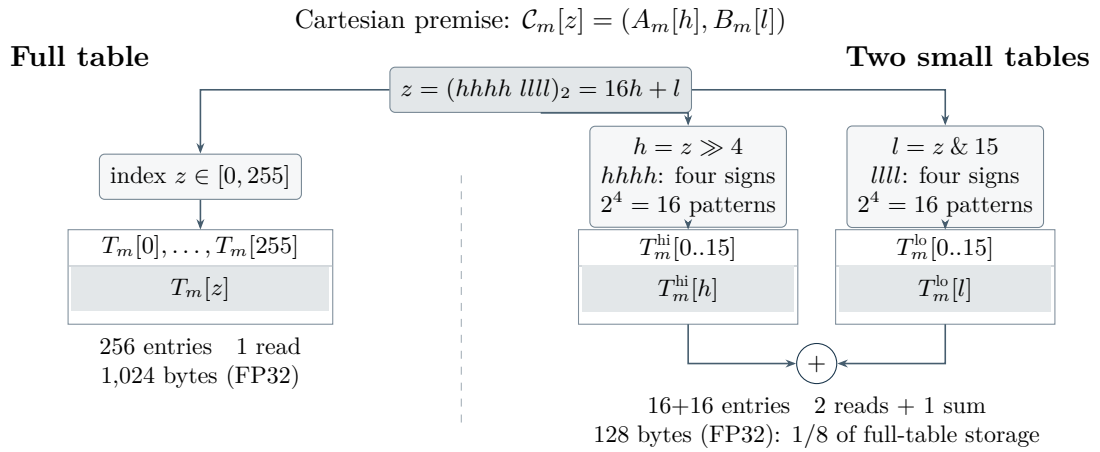
写成 $z = 16h + l$，其中 $h, l \in \{0, \ldots, 15\}$、$\mathcal{C}_m[z] = (A_m[h], B_m[l])$。于是

$$T_m^{\text{hi}}[h] = \|q'_{m,\text{hi}} - A_m[h]\|_2^2, \qquad T_m^{\text{lo}}[l] = \|q'_{m,\text{lo}} - B_m[l]\|_2^2. \tag{5.4}$$

由坐标分离可得

$$T_m[z] = T_m^{\text{hi}}[z \gg 4] + T_m^{\text{lo}}[z \,\&\, 15]. \tag{5.5}$$

存储降为 $16 + 16 = 32$ 项；编码、候选及实数算术分数保持不变（图 5.2）。



Same byte codes and routed candidates; equal scores in real arithmetic.
FP32 evaluation can differ in rounding and ties (T4, 第7.3节).

图 5.2. Cartesian 主码 LUT 分解示意。一个 byte 选择 256 项表中的一项，或用高 / 低 nibble 分别选择两个 16 项表中的一项。坐标分离在实数运算下给出加法恒等式；FP32 舍入可能改变 top-c 和最终 ID。T4 中各表形式的最大相对分数差为 4.64×10^{-7}；Vogue 有 2–4/256 个查询的最终 ID 不同，arXiv / OpenAI-3072 没有差异（第7.3节）。读取和求和描述的是查表表达式。

构造器切分码字 $16h, l$；`verify_split_lut` 检查 nibble / 切片兼容性。任意学习得到的 256 质心 PQ 子码本不一定满足该条件：将编码标签分成 nibbles，并不使质心坐标自动可分。

第7.3节 的同候选 FP32 审计测量不同表形式的数值与排序差异，包括主码保留集合相同后出现的变化。

5.2.2 分组、构造工作量与资源成本

表分组消融将拆分推广到 $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ 个连续坐标组，其 bit 字段与 Cartesian 数字对应。对八位及 $D_s = 8$，每组有 $2^{8/g_{\mathrm{LUT}}}$ 项；每查询的总项数和 FP32 存储为

$$n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = g_{\mathrm{LUT}} 2^{8/g_{\mathrm{LUT}}}, \qquad S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = 4Mn_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \text{ bytes.} \tag{5.6}$$

独立构造表项时，每子空间计算 $D_s 2^{8/g_{\mathrm{LUT}}}$ 个坐标贡献。因此，相较逐项独立构造整表，两组将表存储减为八分之一、坐标工作量减为十六分之一。这是表级成本比，不是应用加速预测。

表 5.2. 八位、八坐标主码子空间的 Cartesian 表权衡。合并加法是连接查得值所需的代数最小值，不含跨子空间累加；不是实测指令计数。

g_{LUT}	项数	FP32 字节	查表数	合并加法	构造项数
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

表先在全局工作区构建；若完整共享内存占用符合查询到的每 block 可选上限 $S_{\max}$，则协作暂存到 shared memory。缓冲容量为 C_{buf} 时，普通 / 分组扫描占用为

$$S_0 = 8C_{\mathrm{buf}} + 8 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}), \qquad S_{\mathrm{grp}} = 8C_{\mathrm{buf}} + 24 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \quad \text{bytes.} \tag{5.7}$$

对已评测路径（不含可选融合精排 / 提前退出临时区），

$$\text{shared lookup} \iff S_{\mathrm{path}} \le S_{\max}.$$

表扫描实验中，$c = 1{,}000$，block 大小为 512 或 1,024，$C_{\mathrm{buf}} = 2{,}048$。整表占 96–384 KiB，两组为 12–48 KiB。NVIDIA 对计算能力 12.0 规定每 block 上限 99 KiB [24]。按此上限，即

使 $M = 96$，整表占用仍超容量（query-centric 路径为 114,696 bytes），而最大两组占用在分组路径中为 65,560 bytes。因此，这些配置下的源码放置规则将整表放在 global memory，将两组表放在 shared memory。这是依据启动规则和公开上限推断的放置；测量记录未保存设备属性查询。

四组和八组每子空间均存 16 项，但查表和合并更多。位提取、依赖、暂存、同步和驻留共同决定权衡。每个扫描组可能暂存一份副本。

16 + 16 nibble LUT 沿用小表先例 [1, 2, 33]。本文通过 GPU 保留 / 精排将其与 JHQ 两级分数集成；实测权衡见 第7.3节。

5.3 打包编码与候选并行内存访问

subspace-major 编码使相邻 lanes / 候选对齐；拆分减少 LUT 存储。

5.3.1 地址映射与四字节打包

列表 ℓ 的行 r 映射为

$$p = o_\ell + r, \qquad \mathrm{offset}(Z[m,p]) = mN + p.$$

固定 m 时，$\Delta\,\mathrm{offset} = \Delta p$。列表边界可能破坏相邻性。

为每个候选打包四个子空间 byte：

$$\mathsf{w}[j,p] = \sum_{t=0}^{3} 2^{8t} Z[4j+t,p], \qquad \mathrm{offset}_{\mathrm{bytes}}(\mathsf{w}[j,p]) = 4(jN+p). \tag{5.8}$$

对 $0 \le t < 4$ 和八位 Z，

$$(\mathsf{w}[j,p] \gg 8t)\ \&\ 255 = Z[4j+t,p].$$

nibbles 索引 式 (5.5)。

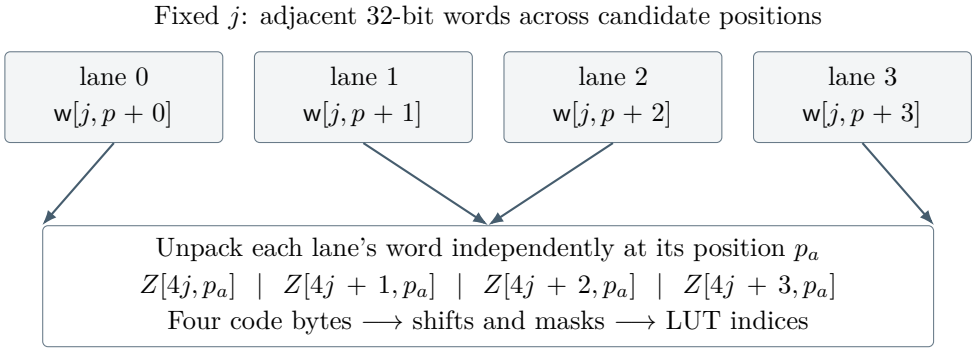


图 5.3. 候选并行的打包访问，展示部分 warp lanes。固定 word group j 时，连续位置访问相邻四字节 word。每个 word 含同一候选的四个子空间码。解包保留各编码 byte；LUT 拆分在其后进行。该图示意地址，不表示实测内存事务次数。

构建将 $[M,N]$ bytes 映射为 $[M/4,N]$ words（$4 \mid M$）。直接类型转换会将不同候选的 bytes 混在一起。编码 byte 成本为

$$R = W = NM, \qquad S_{\text{old+new}} = 2NM.$$

重打包不计入搜索时间。

5.3.2 访存开销与评分一致性

一次 word 载入 / 解包代替四次 byte 载入。逻辑 warp 窗口为 $32 \times 4 = 128$ bytes；事务依赖对齐和活跃 lanes。每候选载荷仍为 M bytes。两条路径均为 subspace-major，因此没有单独分离转置 / 缓存效应。

5.4 多 block 扫描中的精确全局主码 Top-c

各组并行扫描，并合并按计算主码分数保留的候选 [18]。

5.4.1 候选分区与确定性排序

对固定查询，按 probe 顺序串接选定倒排列表，得到 n_c 个候选出现实例。假设每个访问的数据库行仅出现一次，且搜索期间索引固定。对 $g_{\text{scan}} \in \{1, 2, 4, 8\}$，block (q, j) 拥有区间

$$U_j = \left[\left\lfloor \frac{j n_c}{g_{\text{scan}}} \right\rfloor, \left\lfloor \frac{(j+1) n_c}{g_{\text{scan}}} \right\rfloor \right), \qquad j = 0, \ldots, g_{\text{scan}} - 1. \tag{5.9}$$

这里的区间表示对应候选，而非数据库 ID。设备根据路由候选数计算端点。各组大小最多相差一个候选；长列表可能跨组边界。这平衡的是候选数，平均分配列表个数未必能做到。每线程通过 probe 前缀 offsets 找到首个列表，并在扫描自身区间时推进列表游标。本阶段不读残差码。

选择键为

$$K_q(x) = \big(D_{\text{P}}(q, x), p(x)\big), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.10}$$

其中 $p(x)$ 是固定倒排存储中的物理行位置，在该查询候选中唯一，且独立于原子追加顺序。它不是最终结果并列时使用的数据库 ID。非 NaN 分数下，该键定义不同候选的全序。局部选择和归并均使用此顺序。分区独立性还要求各路径计算分数相同。

5.4.2 精确局部保留与全局覆盖

$\text{Top}_c(V)$ 包含前 $\min(c, |V|)$ 个键，其中 $c = \max\{k, \lceil \alpha k \rceil\}$。各组最多保留 c，因为所有全局优胜者都可能位于同组；c/g_{scan} 配额不安全。

引理 5.1 (局部缓冲保留)**.**
令 V_t 为一个 block 扫描到 chunk t 为止已评分的候选，B_t 为该时刻任意压缩或过滤后的有效缓冲。假设完整键固定、追加容量充足且 top-c 压缩精确。令 β_t 为迄今观察到的完整共享阈值的最小值，包括初始顶部哨兵。则

$$\mathrm{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\} \subseteq B_t. \tag{5.11}$$

不共享时，阈值为哨兵，缓冲包含整个前缀 top-c。

证明.
空前缀显然成立。令

$$E_t = \mathrm{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\}.$$

将 V_t 扩展为 V_{t+1} 时，$\beta_{t+1} \preceq \beta_t$，且

$$\mathrm{Top}_c(V_{t+1}) \cap V_t \subseteq \mathrm{Top}_c(V_t).$$

因此，

$$x \in E_{t+1} \cap V_t \Rightarrow x \in E_t \subseteq B_t.$$

局部拒绝的新键已有 c 个前驱，故排名 $> c$。容量可容纳其余符合准入的到达项。对任意符合条件的优胜者 x，

$$\#\{y \in B : K_q(y) \prec K_q(x)\} \leq \#\{y \in V_{t+1} : K_q(y) \prec K_q(x)\} < c,$$

所以精确压缩保留 x。过滤移除 $K > \beta_{\text{seen}}$；运行最小值覆盖陈旧读取。上述步骤通过归纳保持不变量。 □

命题 5.1 (覆盖与截断归并)**.**
在共同全序下固定键，令 U 为已访问空间，V 为任意保留子集。若 $\mathrm{Top}_c(U) \subseteq V \subseteq U$，则 $\mathrm{Top}_c(V) = \mathrm{Top}_c(U)$。成对有序归并并截断为 c 可保持该结果。特别地，对不相交分组，

$$\mathrm{Top}_c\left(\bigcup_j U_j\right) = \mathrm{Top}_c\left(\bigcup_j \mathrm{Top}_c(U_j)\right). \tag{5.12}$$

该陈述也包含候选空间小于 c 的情形。

证明.
每个全局优胜者都在 V 内，而 V 的其他成员均排在全局优胜者之后。因此，从 V 选择可找回它们。对独立分组这一特例，设 x 属于全局 top-c，且位于组 U_j。全局上排在 x 前面的候选少于 c，所以在 U_j 内排在它前面的也少于 c。于是 $x \in \mathrm{Top}_c(U_j)$，局部结果的并集包含所有全局优胜者。取并集前 c 项，即按相同顺序精确找回全局优胜者。若整个空间少于 c 个成员，则全部保留。

归并时，两个有序列表的前 c 项就是并集的 top-c。被丢弃候选在该并集中已有至少 c 个前驱，无法进入任何更大并集的 top-c。对归并树归纳即可得到结论。 □

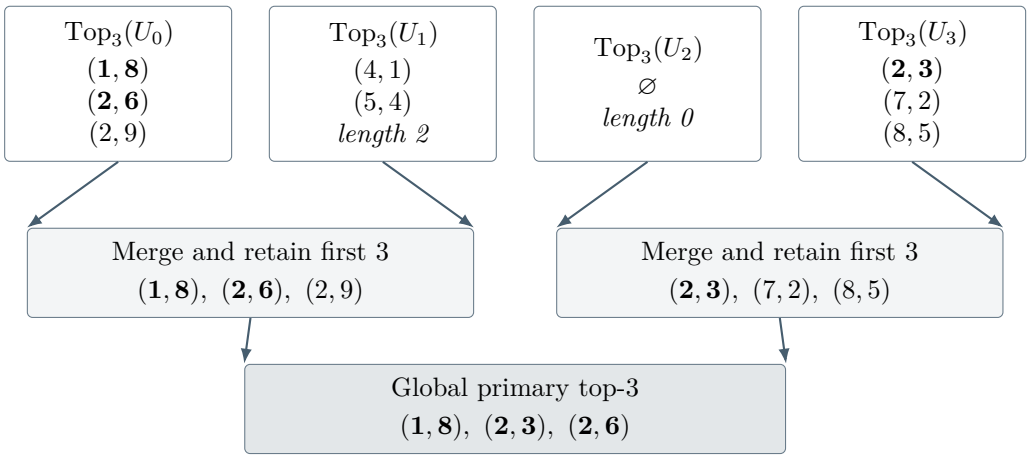


图 5.4. 局部覆盖与截断成对归并，示例取 $c=3$、$g_{\text{scan}}=4$。条目是示意的 (D_{P},p) 键，粗体为全局优胜者。短组 / 空组贡献有效前缀。归并按相同顺序最多保留 c 项。

CUDA 实现维护一个共享内存缓冲，包含 C_{buf} 个键对。其中 C_{buf} 是二的幂；扫描 block 含 T_{blk} 个线程，满足 $C_{\text{buf}} \geq c+T_{\text{blk}}$。bitonic 压缩最多保留 c 个有效键；有效键达到 c 时，阈值设为最后一个键。此前，阈值是高于一切有效键的哨兵。两次压缩间的准入条件为 $K \prec \lambda_j$。阈值具有 c 个见证，所以候选继续到达时，它仍是前缀第 c 个键的上界。

容量由下式保证：

$$|B|+|\Delta V| \leq c+T_{\text{blk}} \leq C_{\text{buf}}.$$

这满足 引理 5.1 的追加前提。结合 命题 5.1与5.2，过滤 / 归并可保留全局优胜者。

显式分数需要 $O(n_c)$ 的写入 / 存储。融合缓冲输出 $O(g_{\text{scan}}c)$ 个键对；寄存器选择是另一种方案 [18]。

5.4.3 归并实现、存储与边界情况

在 $g_{\text{scan}}>1$ 下，256 线程 block 对每个查询进行 $\log_2 g_{\text{scan}}$ 轮归并。每对列表用 Merge Path 切片输出 $\min(c,a+b)$ 项 [14]。所有比较使用 式 (5.10)，轮间以 block 屏障分隔。图 5.4 展示截断不变量。除分区搜索和同步工作外，归并树内部节点合计最多输出 $c(g_{\text{scan}}-1)$ 个键对。

两个交替键对工作区的形状为 $[b_{\text{cap}},g_{\text{scan}},c]$ 与 $[b_{\text{cap}},g_{\text{scan}}/2,c]$：

$$S_{\text{pairs}} = 8b_{\text{cap}}c\left(g_{\text{scan}}+\frac{g_{\text{scan}}}{2}\right) \quad \text{bytes}, \qquad g_{\text{scan}}>1. \tag{5.13}$$

计数 / 阈值另加 $4b_{\text{cap}}g_{\text{scan}}$，以及预分配时的 $8b_{\text{cap}}$ bytes；保留候选 / 其他工作区另计。分配在图捕获前扩容后持续保留。归并共享内存保存长度，而非全部键。

计数界定有效前缀：

$$K_{\text{sort pad}} = (+\infty,\text{INT_MAX}), \qquad K_{\text{output pad}} = (+\infty,-1).$$

有效位置小于 INT_MAX；空组计数为零。负位置被跳过 / 排除。$g_{\text{scan}}=1$ 直接写出。

单 block（$g_{\text{scan}}=0$）按距离精确，截断处并列可任取。有损 `K_LOCAL=4` 需要 `JHQ_EXACT_TOPCK=0`，在 paper-faithful 构建中关闭。

对非空 V，令 $h=\min(c,|V|)$，第 h 个分数为 d_h。仅按距离压缩得到

$$\{x : d_x < d_h\} \subseteq B \subseteq \{x : d_x \leq d_h\}, \qquad |B|=h.$$

每个拒绝项都有 c 个不大于它的见证；并列 ID 可自由选择。T4 通过 CPU 排序 GPU 生成分数，检查这一约束和分组完整键顺序（第7.4.1节）。

5.5 共享完整键阈值与执行调度

局部 top-c 可能为 c 个优胜者输出 $g_{\text{scan}}c$ 个中间条目。共享阈值减少维护工作，调度器随后选择扫描并行度。二者均在固定精排预算下工作；预算选择是 第6章 的独立问题。

5.5.1 具有显式见证的完整键上界

当 $|U| \geq c$ 时，令 $K_{(c)}$ 为全局第 c 个键。若某组持有 c 个不同、已完整评分的候选，其见证最大键 τ_j 满足 $K_{(c)} \preceq \tau_j$。全局空间中已有 c 个候选不晚于 τ_j。共享阈值初始化为顶部哨兵，并通过下式更新：

$$\tau \leftarrow \min_{\prec}(\tau, \tau_j). \tag{5.14}$$

有效候选少于 c 时不发布阈值。若整个空间少于 c，则没有组能发布该阈值，所有有效候选仍可准入。

命题 5.2 (见证阈值下的安全拒绝)**.**
假设键固定，发布值由 c 个不同候选支持，读取返回初始哨兵或某个完整的已发布键。仅在 $K_q(x) \succ \tau$ 时拒绝候选，无论发布时序如何，都不会移除全局 top-c 候选。

证明.
一次发布具有含 c 个不同见证的集合 W_τ：

$$\max_{x \in W_\tau} K_q(x) \preceq \tau \quad \Longrightarrow \quad K_{(c)} \preceq \tau.$$

哨兵读取及已发布键的最小值满足同一不等式。因此

$$x \in \mathrm{Top}_c(U) \implies K_q(x) \preceq K_{(c)} \preceq \tau_{\text{seen}} \implies x \text{ is not rejected.}$$

陈旧读取只会放宽阈值。先前安全拒绝后的发布仍然有效：保留的任意 c 个不同候选仍提供 c 个见证，无论它们是否是该组最佳的 c 个。 □

共享过滤保持全局覆盖（命题 5.1）。准入条件为

$$K_q(x) \prec \lambda_j \quad \text{and} \quad K_q(x) \preceq \tau_{\text{seen}}, \tag{5.15}$$

其中 λ_j 为局部压缩阈值，τ_{seen} 为最后读取的共享键。共享比较保留等号。

5.5.2 发布、过滤与实现限制

对非负 FP32 分数、规范化正零和有效非负位置，

$$\kappa(D, p) = 2^{32}\,\mathrm{uintbits}_{32}(D) + p. \tag{5.16}$$

在此非 NaN 定义域上，浮点 bits 单调性给出

$$(D_1, p_1) \prec (D_2, p_2) \iff \kappa(D_1, p_1) < \kappa(D_2, p_2).$$

有限键排在无穷与全一哨兵之前。无效位置不能发布。

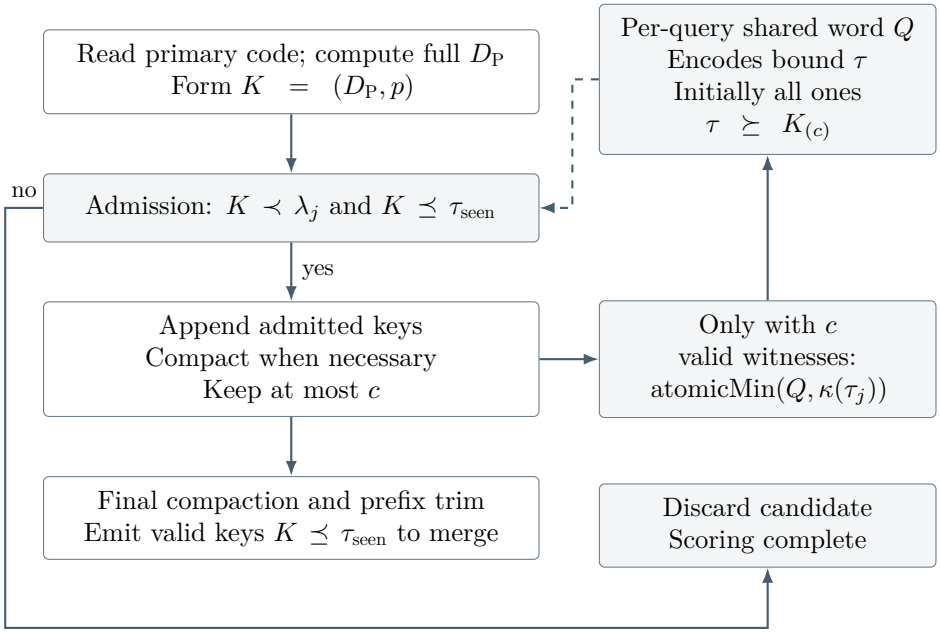


图 5.5. 一个扫描组中的共享完整键过滤。候选在准入前已计算完整主码分数。有效的 c 候选见证集可收紧逐查询阈值；陈旧但完整的阈值只会减弱过滤。最终前缀裁剪减少有效归并输入，但分配的工作区容量不变。完整键读取的前提限定见正文。

每个 chunk 由一个线程轮询。压缩后，c 个有效键允许执行 64 位 `atomicMin`。最终发布 / 搜索裁剪前缀，精确归并处理陈旧额外项。流内重置先于扫描，扫描后归并；组之间从不等待。

完整键前提依赖对齐八字节强 PTX `ld.volatile.global.u64`/`atom.global.min.u64` [26]，超出 C++ `volatile` 自身保证 [25]。CUDA 13.0.88 的 `ptxas`、`sm_120` 提供佐证；没有捕获驱动 JIT 后的 SASS。

过滤发生在完整主码评分后（图 5.5）：维护节省与轮询 / 原子操作开销相互竞争(第7.4节)。

5.5.3 不改变预算地选择并行度

bg_{scan} 个 block 增加并行度，也增加暂存 / 选择 / 归并成本。更小配额需要证明或回退。试验中优胜者集中分布，促成了见证阈值设计（第7.4节）。

定义

$$T_{\mathrm{blk}} = \text{scan threads/block}, \quad H = n_{\mathrm{SM}} T_{\mathrm{resident/SM}}, \quad v = \frac{\mathrm{sd}(n_c)}{\mathrm{mean}(n_c)}.$$

对应规则为

$$g_{\mathrm{fill}} = \max(\{1\} \cup \{h \in \{2,4,8\} : bhT_{\mathrm{blk}} \le 0.6H\}), \tag{5.17}$$

$$g_{\mathrm{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad g_{\mathrm{chosen}} = \max(g_{\mathrm{fill}}, g_{\mathrm{skew}}). \tag{5.18}$$

偏斜规则可覆盖填充上限。评测 harness 使用 512 个数据库向量作为伪查询，在固定路由配置下校准 v，不使用 GT 标签。这不同于第 6 章的抽样输出一致性校准。校准后，调度仅用 batch 大小和存储统计量，无需每次搜索额外主机回读。常数由四个试验配置拟合，再在更多配置上评估。一次性伪查询校准不计入报告的稳态搜索时间。启动耗时观测范围独立于精排

预算校准（第3.5.2节），为

$$T_{\text{gather}} \in [0.81, 7.04]\,\text{ms}, \quad T_{\text{first}} \in [6.00, 27.77]\,\text{ms}, \quad \text{median}(T_{\text{repeat}}) \in [2.72, 18.13]\,\text{ms}.$$

选择一组时，在索引 / probes / 预算固定下分派 $g_{\text{scan}} = 0$。它仅按距离处理并列；要获得相同保留集合，需检查共同排序。集成收益 / regret 见 第7.6节。

5.5.4 观测到的执行路径

对有 170 个 SM、每 SM 48 个 warp 的 RTX 5090 [24]，

$$1000 \cdot 2 \cdot 512 = 1{,}024{,}000 > 0.6 \cdot 170 \cdot 48 \cdot 32 = 156{,}672.$$

因此填充规则选择一组；偏斜可覆盖为四组。

E1 中，9/38 条记录超过 $v = 0.3$，全部为 Vogue（0.338–0.622）。Vogue 的 27 个单元使用四组，105 个 arXiv / OpenAI 单元使用 $g_{\text{scan}} = 0$。后者的收益来自 LUT、打包、lanes 和 probe 选择，不涉及分组保留 / 共享阈值。

Vogue 每 1,000 个查询有 3–15 个纯并列差异，非并列差异为零，最终输出完全相同。128 probes 下，并列范围为 7–11。

观测到的单元数为

	$G = 0$	$G = 2$	$G = 4$
held-out	10	0	2
$S = 128$	2	8	2

十二个单元中有八个换路径。收益取决于负载大小 / 偏斜，不存在固定 batch-256 分界。

启发式未计入驻留、压缩和归并成本；伪查询统计也可能过时。最大实测 regret 为 batch 256 时的 1.217。预算保持固定；第5.6节 处理精排。

5.6 Lane 协作残差评分与最终 Top-k

主码筛选将坐标工作量从 $O(n_c d)$ 改为 $O(cd)$。lanes 在相同预算下协作。

5.6.1 重建评分与查询常数

对 $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$，按实数算术展开：

$$\begin{aligned} D_{\text{H}}(q, x) &= D_{\text{P}}(q, x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) && (5.19) \\ &= 2\|q'\|_2^2 + \|\widehat{p} + \widehat{r}\|_2^2 - 2\langle q', \widehat{p} + \widehat{r} \rangle \\ &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. && (5.20) \end{aligned}$$

该查询常数保持重建距离顺序 [16]。

标量评分先累加残差差值平方，再加主码分数 / 校正项。查询及可放入的码本使用 shared memory。

5.6.2 Lane 到 word 的映射与部分和归约

对 $\ell_{\text{ref}} = L \in \{1, 8, 16, 32\}$，

$$d_{\text{word}} = 32/B_R, \qquad \text{words of lane } a: \ a, a + L, a + 2L, \ldots.$$

一个 word 包含四个八位或八个四位码。蝶形 shuffle 累加 lane 部分和；lane zero 写分数。

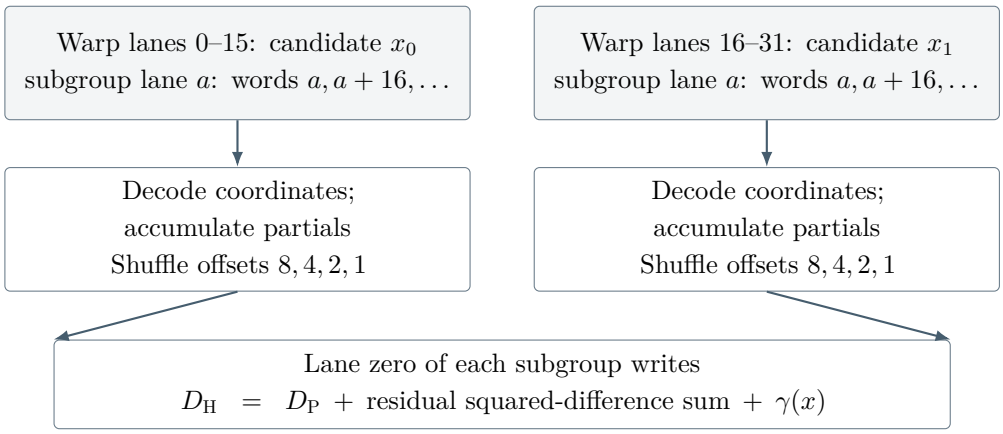


图 5.6. 十六 lane 残差精排。一个 warp 处理两个候选；候选内 lane a 读取残差 words $a, a+16, \ldots$。每个八位编码 word 解码四个坐标。子组求部分和，再加主码分数 / 校正项。候选位置仍可能分散。

行按 word 对齐且满足 $d_{\mathrm{word}} \mid D_s$，保证 word 不越过码本范围。所有 lanes 都参与 shuffle；无效槽位贡献零，随后输出无穷。$L = 16$ 对应每 warp 两个候选。

L0 载入 bytes；L1 载入 words，但保持相同的单 lane 累加。L8 / L16 / L32 用更短的遍历换取 shuffle 和更少的并发候选。FP32 运算结合顺序及局部性会影响结果。

5.6.3 最终选择与数值边界

在至少有 k 个有限、有效且 ID 唯一的保留候选时，每轮计算

$$i^* = \arg\min_{i<c}(D_{H,i}, \mathrm{ID}_i), \quad d^* = D_{H,i^*}, \quad D_{H,i^*} \leftarrow +\infty$$

使用全部槽位，输出 (ID_{i^*}, d^*)：这是保留池内精确的 top-k，没有线程配额。不足额池可能在无穷距离处重复最小有效 ID；全无效池返回无效 ID。测试配置满足有限项前提。

主码并列使用位置，最终并列使用 ID。路由排除 / 量化仍存在；lane FP32 结合顺序可能改变近并列结果。

5.7 IVF 探测列表选择

probe 选择发生在主码扫描前，选择倒排列表，而非数据库向量保留候选。对 IVF 质心 μ_i，选择器排序计算得到的 FP32 值路由分数：

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.21}$$

省略共同查询范数不改变质心顺序。与主码平方距离表不同，这些分数可能为负。输出为

$$n_{\mathrm{sel}} = \min(n_{\mathrm{probe}}, n_{\mathrm{list}}), \qquad I = \mathrm{Top}_{n_{\mathrm{sel}}}^{(s_i,i)}\{0, \ldots, n_{\mathrm{list}} - 1\},$$

并附列表长度前缀 offsets。

OpenAI v69 使用 TF32 旋转 / 质心 GEMM 和 FP32 累加。probe 精确性针对其计算分数；FP32 `fma` 不会消除此前舍入（第7.2.3节）。

5.7.1 Radix 选择与确定性排序

一个 block 进行四趟从最高有效位开始的 radix。对前缀限制直方图 H_t 和从一开始的剩余排名 r_t（$r_0 = n_{\mathrm{sel}}$），

$$b_t = \min\Big\{b : \sum_{a \le b} H_t[a] \ge r_t\Big\}, \qquad r_{t+1} = r_t - \sum_{a<b_t} H_t[a].$$

选中的 bin 扩展前缀。四个 bytes 确定 cutoff / 等键数量。

将带符号零规范化为正零，再按下式映射 bits u：

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.22}$$

对非 NaN 分数，分类可得单调性：

$$\begin{aligned} &D_1 < D_2 < 0 \Rightarrow u_1 > u_2 \Rightarrow \sim u_1 < \sim u_2, \\ &0 \le D_1 < D_2 \Rightarrow u_1 < u_2 \Rightarrow (u_1 \mid 2^{31}) < (u_2 \mid 2^{31}), \\ &D_1 < 0 \le D_2 \Rightarrow F(u_1) < 2^{31} \le F(u_2). \end{aligned}$$

该映射覆盖无穷 / 零，不同于仅适用于非负值的 式 (5.16)。重复使用 `fma` 固定重算键。

直方图存储为

$$256 \text{ bins} \times 32 \text{ lane replicas} \times 4 \text{ bytes} = 32\,\text{KiB}.$$

复制减少同 warp 冲突，但仍有 warp 间竞争 / 归约成本。

令 h 为 cutoff 键。收集阶段形成

$$A_< = \{i : F(u_i) < h\}, \quad A_= = \{i : F(u_i) = h\}, \quad m = n_{\text{sel}} - |A_<|.$$

取 $A_<$ 及 $A_=$ 中 ID 最小的 m 项，使用有序 ballot / 前缀计数，而非原子到达顺序。对所得集合 A，

$$\text{rank}(i) = |\{j \in A : (s_j, j) \prec (s_i, i)\}|, \qquad \text{out}[\text{rank}(i)] = i.$$

选定长度产生前缀 offsets。对超出数量的 probes，

$$i_j = i_{n_{\text{sel}}-1}, \quad L_j = 0 \quad (j \ge n_{\text{sel}}), \qquad \omega_{j+1} = \omega_j + L_j.$$

零长度避免重复扫描。要求 $n_{\text{list}}, n_{\text{probe}} > 0$。

5.7.2 成本与独立正确性约束

直方图 / 收集重新读取分数；排名成本为 $O(n_{\text{sel}}^2)$，共享内存随之增长。小规模选择也需支付固定直方图存储；这是确定性集成，不是统一的线性成本。

测试覆盖并列、零、无穷、极值和超量 probes（第7.4节）。

FP16 近似和跨进程检查另行处理（第7.4节）。

第 6 章工作负载级预算校准

与较大预算参考的无标签一致性，用于为后续 batches 选择一个倍率。复用需在所需质量下收回校准成本。

6.1 精排预算与搜索成本

对路由集合 $U(q)$，精排倍率设置为 [16]

$$c(\alpha) = \max\{k, \lceil \alpha k \rceil\}, \qquad C_\alpha(q) = \mathrm{Top}^{\mathrm{primary}}_{\min\{c(\alpha), |U(q)|\}} U(q). \tag{6.1}$$

分析要求 $|C_\alpha| \geq k$；不足额槽位填哨兵。预算改变保留数，probes 改变 U。

当 $k = 10$ 时，网格保留 20–2,000 个候选，无需重新训练。更低预算节省读取量，不减少残差存储或已预留工作区。

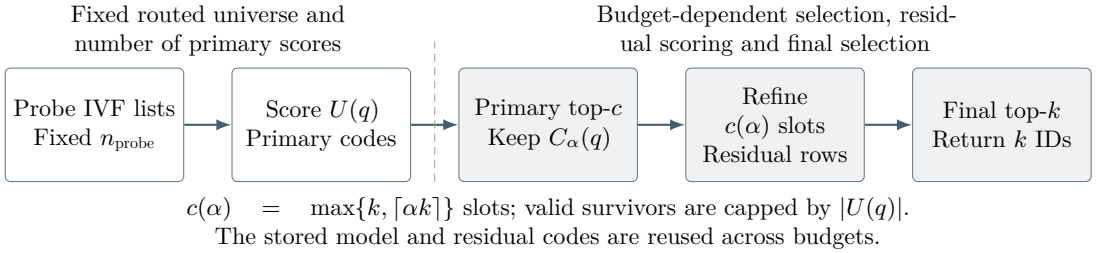


图 6.1. 精排预算改变主码分数生成后的工作。在查询、索引及 probe 数固定时，路由候选空间不变。高亮阶段依赖保留容量；这是逻辑工作分解，不是实测延迟分解。扫描内核也执行依赖预算的准入和压缩。候选少于 c 时需要补位。

在 batch / 路由固定时，记

$$T(\alpha) = T_{\mathrm{base}} + T_{\mathrm{select}}(c(\alpha)) + T_{\mathrm{refine}}(c(\alpha)) + T_{\mathrm{final}}(c(\alpha)). \tag{6.2}$$

T_{base} 包含传输、旋转、路由和主码算术（图 6.1）。选择 / 资源转换使延迟不单调；规则最小化的是被接受预算（第7.5.5节）。

试验复用索引，但重新执行完整搜索，包括旋转、probes 和主码评分；校准成本不只是残差工作。

在自适应 ANN 方法中的定位。 α 对应 Faiss `k_factor` 和 cuVS 精排中的候选扩展作用 [7, 10]。其他方案使用 GT 调参 [9]、学习式逐查询停止 [19] 或预测召回率 [6]。本文用无标签一致性选择一个可复用的工作负载预算。分数仍来自重建向量，不同于 cuVS 使用原向量精排的示例。

6.2 输出一致性与预算选择

设 $\mathcal{R}_\alpha(q)$ 为固定索引上预算 α 返回的 k 个有效且互异数据库 ID 的集合。定义 $\mathcal{R}_{\mathrm{ref}}(q) = \mathcal{R}_{\alpha_{\mathrm{ref}}}(q)$。降序网格为

$$A = (200, 100, 64, 32, 16, 8, 4, 2), \qquad \alpha_{\mathrm{ref}} = A[0] = 200. \tag{6.3}$$

对 S 个校准查询 $\mathcal{Q}_S$，定义

$$\begin{aligned} D_S(\alpha) &= \sum_{q \in \mathcal{Q}_S} \left(k - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{\mathrm{ref}}(q)|\right) \\ &= \frac{1}{2} \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_\alpha(q) \triangle \mathcal{R}_{\mathrm{ref}}(q)|. \end{aligned} \tag{6.4}$$

当且仅当 $D_S(\alpha) \le \epsilon = 1$ 时接受：忽略顺序 / 距离，最多缺一个参考 ID。有限网格目标为

$$\widehat{\alpha} = \min\{\alpha \in A : D_S(\alpha) \le \epsilon\}. \tag{6.5}$$

参考预算是回退选择。第6.3节 给出精确二分的条件。

当 $k = 10$ 时，$\epsilon = 1$ 得到

S	32	64	128	500
$\epsilon/(Sk)$	0.003125	0.0015625	0.00078125	0.0002

更大样本改变覆盖与严格程度。独立留出查询用于评估同一有限池抽样的结果（第7.5节）。

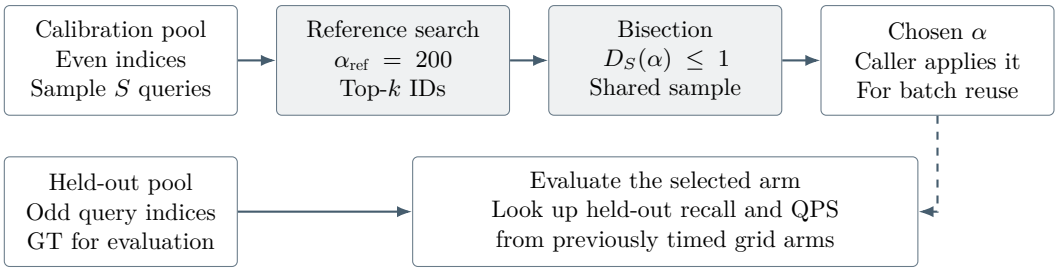


图 6.2. 工作负载级校准及其评估。预算决策使用参考 ID；GT 评估留出质量。所选实验臂计时来自固定臂测量。实现边界见 第3.5.2节。

6.2.1 一致性控制什么

在 命题 6.1 条件下，不一致数等于被主码筛选排除的参考答案数：

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_{\text{ref}}(q) \setminus C_\alpha(q)|. \tag{6.6}$$

对含 k 个元素的输出与 GT 集合 $\mathcal{T}_k$，集合差给出

$$|\mathcal{R}_{\text{ref}} \cap \mathcal{T}_k| - |\mathcal{R}_\alpha \cap \mathcal{T}_k| \le |\mathcal{R}_{\text{ref}} \setminus \mathcal{R}_\alpha|.$$

求和再除以 Sk，得到相对界。在同一样本上，

$$\begin{aligned} r_{\mathcal{Q}_S}(\alpha) &\ge r_{\mathcal{Q}_S}(\text{ref}) - \frac{D_S(\alpha)}{Sk}, \\ D_S(\alpha) \le \epsilon \implies r_{\mathcal{Q}_S}(\alpha) &\ge r_{\mathcal{Q}_S}(\text{ref}) - \frac{\epsilon}{Sk}. \end{aligned} \tag{6.7}$$

范围。 该界是相对参考、限定在本样本上的界。GT 评估测量参考剩余误差（第7.8.2节）。

6.2.2 抽样分辨率与有序检验

令 H_S 计数缺失任意参考 ID 的查询。因为 $H_S \le D_S$，接受意味着 $H_S \le 1$。对预先指定预算和 i.i.d. 查询，容许一次事件的单侧 95% 二项上限满足 [22]

$$(1-u)^S + Su(1-u)^{S-1} = 0.05. \tag{6.8}$$

依赖模型的分辨率为

	$S = 128$	$S = 500$
i.i.d. query events	0.03652	0.009452
i.i.d. homogeneous result slots	0.003701	0.0009484

更强的结果槽位模型给出约 $4.74/(Sk)$。查询 i.i.d. 不意味着槽位独立。这两个模型都不认证固定池抽样。

只有逐实验臂检验有效时，单调性才允许固定序列控制 [3]。本文的一槽位阈值测量一致性，不认证 GT 风险。

有限池扩展。 假设每查询 $X_q(\alpha)$ 在该执行区间内固定，与子集 / 位置无关。池大小为 N_{cal}，计数为 $n_j = |\{q : X_q = j\}|$，均匀无放回抽样仅在全零样本或恰有一个单计数查询时接受 [23]：

$$\Pr\{D_S(\alpha) \le 1\} = \frac{\binom{n_0}{S} + n_1\binom{n_0}{S-1}}{\binom{N_{\text{cal}}}{S}}. \tag{6.9}$$

越界二项系数取零。预测 GT 风险还需网格接受结构和留出固定臂质量。记录的子集和对这些条件进行经验检查（第7.5.5节）。

6.3 预算搜索与单调性条件

6.3.1 二分与完整枚举

记 $J = |A|$。实现先计算一次样本参考结果，再在降序网格中寻找最大的可接受索引：

1. 设置 $l = 0$、$h = J - 1$。零是回退项。
2. 当 $l < h$ 时，运行 $A[j]$ 并计算 $D_S(A[j])$，以测试 $j = \lfloor (l + h + 1)/2 \rfloor$。
3. 若 $D_S(A[j]) \le \epsilon$，设置 $l = j$；否则设置 $h = j - 1$。
4. 返回 $A[l]$，复用前应用它。

八点网格除参考搜索外，最多需要三次完整候选搜索。调用 / 计时见 第3.5.2节。

枚举在相同样本上重跑参考及七个实验臂。它能处理接受集合中的空洞；二分只有在接受索引构成前缀时才能找到其最小预算。两种计时都包含参考搜索。

6.3.2 单调性的一个充分条件

命题 6.1 (嵌套保留集合保持参考成员资格)**.**
对一个查询，假设路由固定，且每当 $\alpha \le \beta \le \alpha_{\text{ref}}$ 时保留集合满足 $C_\alpha \subseteq C_\beta \subseteq C_{\alpha_{\text{ref}}}$。假设至少有 k 个有效保留候选、精排分数固定、最终并列使用同一确定性全序。各结果均是该顺序下精确 top-k。则

$$|\mathcal{R}_\alpha \cap \mathcal{R}_{\text{ref}}| = |C_\alpha \cap \mathcal{R}_{\text{ref}}|, \qquad D_S(\alpha) \ge D_S(\beta). \tag{6.10}$$

当上述条件对每个抽样查询都成立时，后一陈述成立。

证明.
对 $x \in \mathcal{R}_{\text{ref}} \cap C_\alpha$，共同全序给出

$$|\{y \in C_\alpha : y \prec x\}| \le |\{y \in C_{\alpha_{\text{ref}}} : y \prec x\}| < k.$$

故 $x \in \mathcal{R}_\alpha$。结合 $\mathcal{R}_\alpha \subseteq C_\alpha$，得到 式 (6.10) 中的交集恒等式。由嵌套性可得 $|C_\alpha \cap \mathcal{R}_{\text{ref}}| \le |C_\beta \cap \mathcal{R}_{\text{ref}}|$。对查询的缺失参考计数求和，得到 $D_S(\alpha) \ge D_S(\beta)$。 □

固定键下精确主码 top-c 产生嵌套；固定键精排进一步产生单调一致性。这支持二分，但不意味着池扩大时 GT 召回率或运行时间单调。

Case	C_1	C_2	C_3	Reference overlap
Nested	$\{b\}$	$\{b, c\}$	$\{a, b, c\}$	0, 0, 1
Non-nested	$\{a\}$	$\{b, c\}$	$\{a, b, c\}$	1, 0, 1

图 6.3. 构造的 $k = 1$ 示例，精排顺序为 $a \prec b \prec c$，参考为 $\{a\}$。嵌套情况保持参考成员资格。当主码分数全部并列时，不同并列选择可产生非嵌套中间池，破坏单调性。这些是解释性集合，不是实测 GPU 输出。

6.3.3 已评测分组路径的推论

同次抽样内，S、路由和扫描线程固定；`scheduled_groups(S)` 不依赖 c。分组预算使用相同 LUT 求和。固定 B_r、16 lanes 及最终 $(D_{\mathrm{H}}, \mathrm{ID})$ 顺序，使候选分数固定。

推论 6.2 (分组校准)**.**
对已评测分组配置，在第 4、5 章的表示与共享阈值约束下，各预算的主码保留集合嵌套。因此 命题 6.1 适用，二分与枚举返回相同的网格最小接受预算。

证明.
引理 5.1与命题 5.1与5.2 在共同键 $(D_{\mathrm{P}}, \mathrm{position})$ 下给出精确主码 top-c。其前缀随 c 增大而嵌套。固定精排 / 最终顺序满足 命题 6.1 的其余前提。 □

推论覆盖 2,176 次抽样；128 次 OpenAI-3072 / $S = 128$ 抽样仅按距离选择。记录的全部 2,304 条曲线均单调并与枚举一致（第7.5.5节）；这是有限实验证据。

全池覆盖。 两个 FULL 单元对 Vogue 分组，十个使用 $g_{\mathrm{scan}} = 0$。十二个均与枚举一致。十个仅按距离的单元不在 推论 6.2 范围内：嵌套需要共同 cutoff 顺序，或没有跨截断并列。校准与留出使用相同路径并不能保证这一点。

基于排名的扩展。 在同样的嵌套 / 固定分数前提下，从一开始的主码排名给出

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \sum_{x \in \mathcal{R}_{\mathrm{ref}}(q)} \mathbf{1}\{\mathrm{rank}_{\mathrm{P}}(x) > c(\alpha)\}. \tag{6.11}$$

导出参考排名可以替代重复预算搜索。这尚未实现。

6.4 预算选择基线

表 6.1 区分预算选择与评估。固定实验臂共享索引、路由和留出查询；调用和计时细节见 第3.5.2节。

表 6.1. 预算对照及回答的问题。留出 GT 用于评估每个结果，但只有诊断 oracle 使用它选择预算。一致性实验臂的参考固定为 200。

对照	选择信息	问题
固定 100	不校准	相对继承默认值的收益?
固定 64 及更小网格臂	不校准	简单固定选择能否消除校准需求?
抽样二分	S 个无标签查询；槽位一致性	低成本样本能否选出可复用预算?
同样本枚举	相同 S、参考及阈值；全部网格臂	二分是否漏掉网格最小接受预算?
全池校准	全部 500 个校准查询；相同二分规则	更广覆盖及更严格归一化容许量带来什么变化?
留出 Oracle	全部臂的 GT 召回及计时 QPS	哪个已测实验臂在诊断质量约束下最快?

对留出召回率 / QPS $\widehat{r}, \widehat{Q}$ 及 $\widehat{r}^* = \max_{\alpha \in A} \widehat{r}(\alpha)$，

$$\alpha_{\mathrm{oracle}} \in \arg\max_{\alpha \in A} \left\{ \widehat{Q}(\alpha) : \widehat{r}(\alpha) \geq \widehat{r}^* - \delta \right\}, \qquad \delta = 10^{-3}. \tag{6.12}$$

这一有标签 / 计时的诊断使用召回容差 δ，而非槽位容许量 ϵ。

FULL 在无标签下收紧 $\epsilon/(Sk)$；枚举检查搜索正确性。一致性最小化接受预算，oracle 则最大化质量约束下的速度。

固定 64/200 分别测试更小 / 更高预算（第7.5节与表 7.9）。

6.5 校准质量与复用成本

6.5.1 选择校准规模

较强的独立槽位模型提示

$$\frac{4.74}{Sk} \simeq 10^{-3}, \quad k = 10 \quad \Longrightarrow \quad S \simeq 474 \approx 500.$$

这分辨的是参考不一致，不是 GT 风险（第6.2节）。FULL 质量与抽样风险见 表 7.8与图 7.6。

FULL 仍需满足 第6.3节 的路径 / 排序条件。

6.5.2 风险与所选实验臂性能

在 m 次校准抽样中，令第 r 次选择 $\widehat{\alpha}_r$。定义留出损失与越阈频率为

$$L_r = \widehat{r}^* - \widehat{r}(\widehat{\alpha}_r), \qquad \widehat{p}_\delta = \frac{1}{m}\sum_{r=1}^{m} \mathbf{1}\{L_r > \delta\}. \tag{6.13}$$

平均损失、越阈频率和尾部大小回答不同问题。表 7.9与图 7.6 报告存档汇总；补充逐抽样最大损失见 第7.5.5节。

报告的所选实验臂速率为 $\overline{Q} = m^{-1}\sum_r \widehat{Q}(\widehat{\alpha}_r)$，由已计时固定臂查表得到。它不同于中位预算的速率，也不同于校准后查询流的吞吐。质量与速度必须结合阅读（表 7.10）。

6.5.3 复用成本

对后续 B_{reuse} 个等大小 batches，建立模型

$$T_{\text{policy}}(B_{\text{reuse}}) = T_{\text{cal}} + B_{\text{reuse}}T_{\text{chosen}}, \qquad T_{\text{fixed}}(B_{\text{reuse}}) = B_{\text{reuse}}T_f. \tag{6.14}$$

当 $T_{\text{chosen}} < T_f$ 时，回本次数为

$$B^*_{\text{reuse}} = \frac{T_{\text{cal}}}{T_f - T_{\text{chosen}}}. \tag{6.15}$$

确有

$$\begin{aligned} T_{\text{policy}} < T_{\text{fixed}} &\iff T_{\text{cal}} < B_{\text{reuse}}(T_f - T_{\text{chosen}}) \\ &\iff B_{\text{reuse}} > B^*_{\text{reuse}} \qquad (T_{\text{chosen}} < T_f). \end{aligned}$$

若 $T_{\text{chosen}} \geq T_f$，则没有吞吐回本。batch 时间为 $T = 1000n/Q$ 毫秒。选择变化时，凸性给出 $1/\mathbb{E}[Q] \leq \mathbb{E}[1/Q]$：平均 QPS 的倒数低估平均时间。边界 / 直接检查见 第3.5.2节与第7.5.6节，该 campaign 的估计见 表 7.10。

周期校准为每个 batch 增加 T_{cal}/H。自动漂移检测 / 更新留待后续工作。

6.6 校准复用与适用范围

复用需要稳定的索引 / k / 路由 / 查询（第6.3节）；质量、尾部和成本见 第6.5节。漂移检测尚未实现。

调用方必须在复用前显式应用 α。

预算证据使用 v69 / OpenAI TF32；FP32 前沿检查没有重新校准预算。

第 7 章实验评估

本文测试构建 / 容量 (RQ1)、执行 (RQ2)、预算质量 / 复用 (RQ3) 和竞争区间 (RQ4)。

7.1 实验协议

7.1.1 比较协议与可复现性

第3.2节、第3.3节与第3.5节 定义协议。搜索不计构建 / 校准；各消融使用自己的基线。

CPU JHQ 的设置 / 协议未匹配：十月 Vogue 使用 1,024 个列表、$[8, 4]$ 位，而 E1 为 4,096 个列表。因此没有可匹配的 CPU / GPU 加速比。

自适应调用 / 仅设备计时见 第3.5.2节。

指标与前沿估计器遵循 第3.2节与第3.4节；g_{LUT} 和 g_{scan} 保持 第5.1节 中不同的含义。

7.2 跨系统召回率与吞吐比较

7.2.1 比较范围与前沿构造

图 7.1 在 表 3.2 的前四个数据集上，将 JHQ-GPU 与 第3.4节 的图、压缩 IVF 和暴力搜索基线比较 [17, 28, 33]。E1 使用 batch 1,000 和共同 GT；T5 覆盖小 batch（第7.2.6节），内部消融见 第7.6节。

E1 将 105 个 arXiv / OpenAI 单元分派到 $g_{\mathrm{scan}} = 0$，27 个 Vogue 单元因变异系数（coefficient of variation，CV）为 0.338–0.622 分派到 $g_{\mathrm{scan}} = 4$。只有 Vogue 使用分组保留、共享阈值和归并（第5.5节）。

E1 有 1,590 个有效点：JHQ 每数据集 27–39 个，各基线家族 54–264 个。未出现失败、无效 ID 或重复行标记。参数扫描共享索引（第3.4节）。

表 3.3与第3.4节 定义有限覆盖，超出目标未测量。

存储不同：九位 RaBitQ 的名义编码字节为 $9d/8$；4/8 位码更少。表 4.1与第7.7节 在性能包络之外给出表示成本和实测分配。

7.2.2 相同召回率下的吞吐

表 7.1 使用 第3.4节 定义的插值和满足阈值的实测估计器报告召回率 0.97。

表 7.1. 召回率 0.97 的比较。每个比值为“插值 / 实测”；大于一有利于 JHQ。JHQ kQPS 为插值估计。arXiv 与 RaBitQ 的小幅差异随估计器改变方向。

数据集	JHQ kQPS	相对 CAGRA fp32	相对 IVF-RaBitQ
Vogue	230.8	1.416 / 1.537	1.059 / 1.046
arXiv	258.6	1.317 / 1.422	0.996 / 1.015
OpenAI-1536	126.7	1.747 / 2.294	1.130 / 1.111
OpenAI-3072	55.9	1.747 / 2.088	1.157 / 1.346

在索引、probes、预算及最终输出固定、batch 1,000 下，去掉分组使 Vogue 在召回率 0.97 时的 JHQ / RaBitQ 比从 1.059 降至 0.933。这分离的是集成分组路径，不只是共享阈值。

CAGRA int8 在 0.90–0.92 领先 1.20–1.98×，但不覆盖 0.97。RaBitQ 赢得部分 0.98+ 目标；CAGRA fp32 在 arXiv 接近 0.99 时领先（图 7.1）。

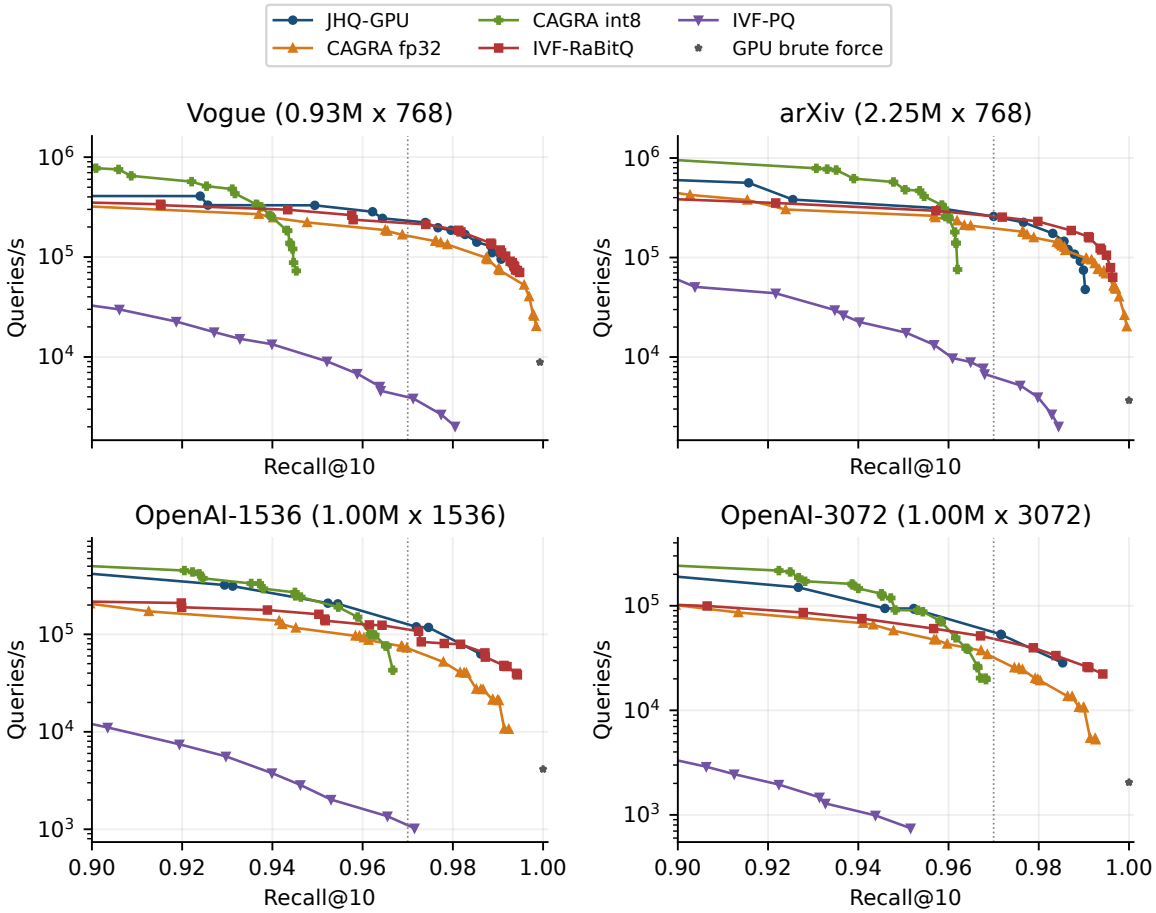


图 7.1. 四个 E1 数据集上的实测主机到主机召回率—QPS 前沿。显示范围从 0.90 开始，0.97 处画垂线。包括 CAGRA int8 在内的所有基线家族均参与前沿构造。线连接实测点；吞吐轴为对数。

7.2.3 OpenAI 数据集上的精度敏感性

补充源码检查确定，虽然输入 / 存储为 FP32，且源码注释有误导，E1 实现对 OpenAI 旋转和质心 GEMM 使用 TF32 乘法与 FP32 累加（第4.2节）。主 E1 前沿仍是该实现的历史测量。campaign D 中，独立 FP32 变体修改 cuBLAS 运算模式开关，在 batch 1,000 下重放 37/39 个 OpenAI 配置，每个五轮。不同 probe 设置间交替变体执行顺序。两者复用同一 TF32 训练模型；干预改变 add / 查询算术，包括数据库编码与路由，并非比较独立 FP32 训练的模型。表 7.2 汇总成对变化。

表 7.2. FP32 敏感性。QPS 比为相同配置下 FP32 / TF32 的几何平均；召回差采用同一方向。末两列比较召回率 0.97 下的 FP32 前沿与存档 E1 IVF-RaBitQ、CAGRA fp32 前沿。campaign D 未重测基线。

数据集	配置对	QPS 比	Δ 召回	相对 RaBitQ	相对 CAGRA
OpenAI-1536	37	0.953	+0.0003–+0.0005	1.086	1.679
OpenAI-3072	39	0.951	+0.0000–+0.0003	1.079	1.629

相同配置下，FP32 吞吐降低约 5%，召回增加 0–0.0005。召回率 0.97 时，FP32 比同会话 TF32 慢约 3%，但仍为存档 RaBitQ 的 1.086/1.079×、存档 CAGRA fp32 的 1.679/1.629×。新会话 v69 对存档 RaBitQ 为 1.118/1.114，历史为 1.130/1.157。D 对基线的比较跨会话；仅 v69 / v72 同会话。图 7.2 展示两条 JHQ 前沿与存档对照。T1 单独重测同会话锚点，网格敏感结果见 第7.2.4节。

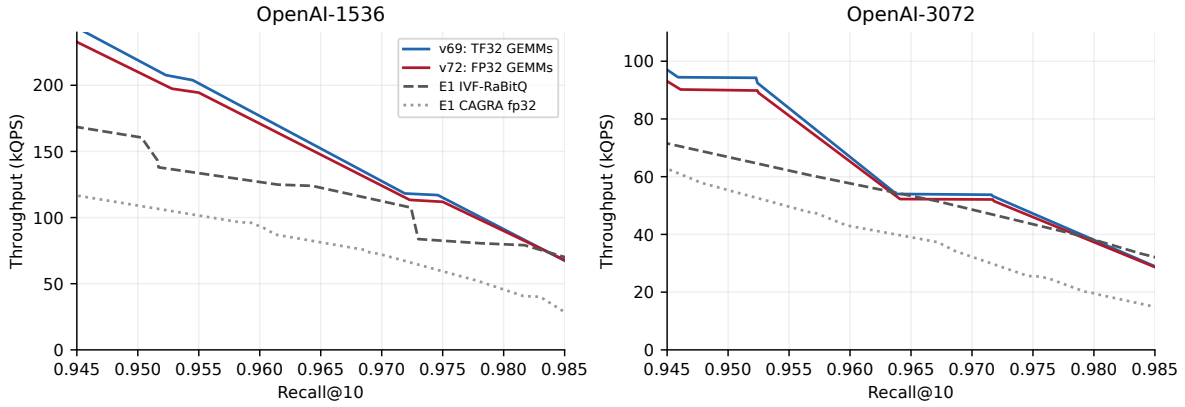


图 7.2. campaign D 的精度前沿。实线为同会话 JHQ 变体；虚线 / 点线基线为历史 E1 测量。均使用本章前沿与插值规则。

7.2.4 同会话基线比较

T1 使用 E1 内存池，每个 OpenAI 数据集十二个成对配置。基线运行前后各有一次五轮 JHQ 重复；中位数以几何方式组合。相同配置 FP32 / TF32 均值为 0.956/0.964。两者均复用 TF32 训练缓存，改变 add / 查询算术。

表 7.3. T1，召回率 0.97、batch 1,000。基线比值为插值 / 严格实测阈值；FP32 / TF32 为插值。所有方法处于同一会话，不同于 D 的存档基线。

数据集	FP32/TF32 at 0.97	FP32 / RaBitQ	FP32 / CAGRA
OpenAI-1536	0.980	1.126 / 1.053	1.752 / 1.633
OpenAI-3072	0.973	1.328 / 1.316	1.825 / 2.019

OpenAI-3072 的 RaBitQ 括区间 0.9669–0.9796 召回体现网格敏感性。这些锚点不替代 E1，也不提供 FP32 重训练结果。

7.2.5 解释与比较限制

两个 OpenAI 数据集均使用 Qdrant DBpedia 嵌入 [31, 32]。截断 / 重新归一化后，1,000 个查询行与 10,299 个抽样基础行的余弦为 ≥ 0.9980，中位数 1.000000，GT 重合度 9.046/10。没有实体 ID，这些检查支持行序对齐。维度翻倍后，CAGRA 比值仍为 1.747，RaBitQ 为 1.130/1.157，因此仅凭维度无法解释。

收益伴随更小的 128-probe 预算锚点：OpenAI 为 4–8，Vogue 为 64，arXiv 为 100，后续 FULL 为 200。这是相关观察，不是独立效应，也不是匹配前沿的参数设置。

第7.8节 检查小幅领先的敏感性，以及 第3.3节 中接口、显存和编译差异的影响。

7.2.6 小 Batch 改变竞争区间

T5 在四个 E1 数据集、batch 8/32 上比较调度 JHQ、CAGRA fp32 / int8 和 IVF-RaBitQ。五轮同步主机到主机测量分别使用 1,000/992 查询。1,808 个测点包含 JHQ 单 block 对照，不含 IVF-PQ / 暴力搜索。图 7.3与表 7.4 比较所得区间。

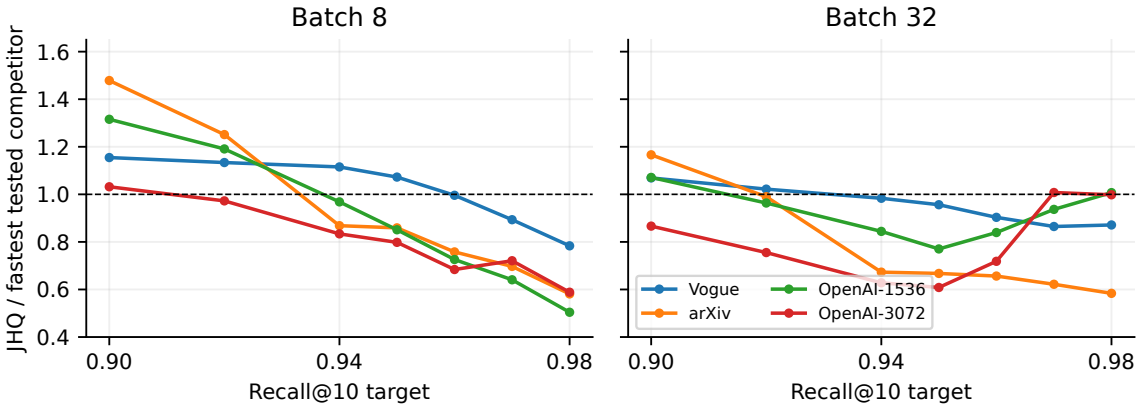


图 7.3. T5 的 JHQ / 最快对手 QPS 比，采用保留合格实测端点的插值。大于一有利于 JHQ。严格阈值见 表 7.4。

表 7.4. T5 QPS 比：保留合格端点的插值 / 严格实测阈值。各估计器独立选择最快对手；满足阈值的更高召回端点可入选，无需外推。

数据集	b	$R = 0.90$	$R = 0.95$	$R = 0.97$
Vogue	8	1.155 / 1.091	1.073 / 0.980	0.893 / 0.854
Vogue	32	1.069 / 0.997	0.956 / 0.895	0.865 / 0.839
arXiv	8	1.479 / 1.454	0.859 / 0.851	0.697 / 0.698
arXiv	32	1.166 / 1.148	0.667 / 0.661	0.622 / 0.582
OpenAI-1536	8	1.316 / 1.131	0.851 / 0.825	0.641 / 0.620
OpenAI-1536	32	1.071 / 0.915	0.770 / 0.752	0.937 / 0.914
OpenAI-3072	8	1.032 / 0.953	0.798 / 0.798	0.721 / 0.702
OpenAI-3072	32	0.866 / 0.734	0.608 / 0.771	1.008 / 1.351

batch 8 / 召回率 0.97 时，CAGRA fp32 在两种估计器下均赢得全部四个数据集，JHQ 比为 0.641–0.893。batch 32 时比为 0.622–1.008；实测阈值下 OpenAI-3072 有利于 JHQ。Vogue 的 batch-8 / 0.95 领先需要插值。网格分辨率会改变接近持平的结果。

纳入合格端点时，召回率 0.90 比值为 batch 8 的 1.032–1.479、32 的 0.866–1.166；严格实测下 JHQ 分别赢三 / 一个。省略端点会夸大收益。调度在 0.95 处相对单 block 提升 1.7–3.3×，但不保证跨系统获胜。

空调用中位数为 0.012–0.072 ms，仍计入总时间。CAGRA 使用 Python，JHQ / RaBitQ 使用 C++；预热不同（第3.5.2节）。

7.3 Cartesian LUT、打包载入与 Lane 精排

RQ2 消融同时测量内核与应用效应。

7.3.1 LUT 分解与吞吐

LUT 消融测试 第5.2节 成本模型与 FP32 排序效应。

图 7.4 比较六个数据集上的 Cartesian 表分组，probe 数为 8/32/128/512，固定 $\alpha = 100$，$g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$。五轮使用 batch 1,000，Stella 为 999。两组在 22/24 单元领先，例外接近持平：arXiv / 128 的一组快 0.23%，OpenAI-3072 / 8 的四组快 0.84%。两组对一组成对获胜 23/24，全部数据集几何平均大于一，范围 1.021–1.318。共享索引单元并非独立试验。

两组 / 一组比值从 0.998 到 1.914。四组和八组均存 16 项，但需要更多查表与加法。这说明表更小不一定提高吞吐；占用率和指令成本未被单独分离。任意四臂单元中报告的最大召回差为 10^{-4}。128 probes、$\alpha = 100$ 下，T4 在 Vogue、arXiv 和 OpenAI-3072 上比较相同

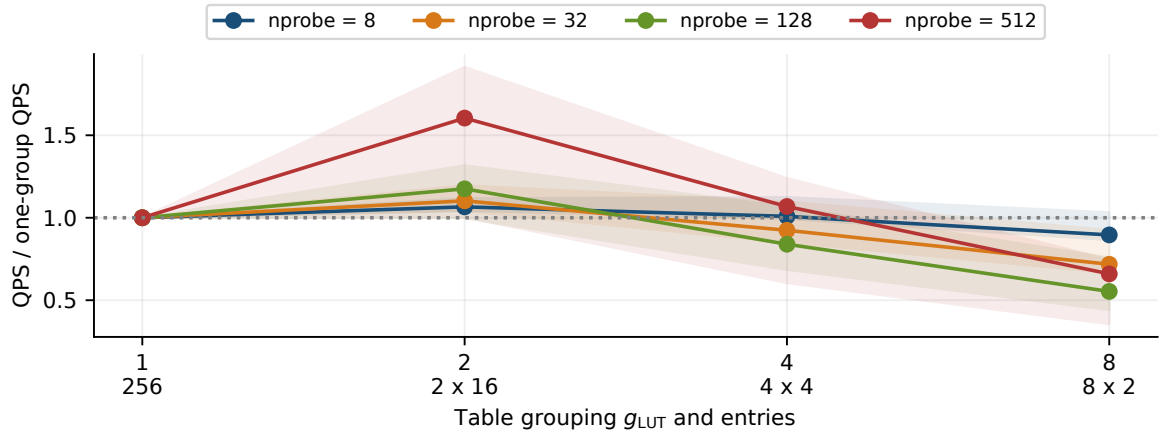


图 7.4. 固定 $\alpha = 100$ 下的 Cartesian 表分组（v71）。组数表示 g_{LUT}，不是并行扫描组。曲线为各数据集相对一组实验臂 QPS 比的中位数；阴影表示范围。

候选。整表与 16 + 16 表有 22.6–26.1% 的候选分数 bit 模式不同，最大相对误差 4.53×10^{-7}。跨全部形式，20.5–29.4% 不同，最大相对误差 4.64×10^{-7}。Vogue 有 2–4/256 个最终 ID 差异，arXiv / OpenAI 没有。主码保留集合不变时，最终 ID 仍可变化，因为舍入后的主码分数进入精排（第3.5.2节）。

7.3.2 打包访问与协作残差评分

打包在 48 对匹配配置中获胜 1.012–1.658×，覆盖六个数据集、四个 probe 数及 $g_{\mathrm{LUT}} \in \{1, 2\}$，召回差最多 10^{-4}。两个布局均为 subspace-major。打包 / 载入 / 解包一起改变。每臂仅一个汇总计时，比五轮 v71 的计时证据少。

图 7.5与表 7.5 进一步区分残差精排中的宽载入和 lane 协作。L0 是原始标量路径；L1 只改变载入路径，不把一个候选分到多个 lanes；L16 为每候选分配 16 lanes。同一进程在五轮计时中轮换实验臂顺序，每轮每臂三次搜索。十二个普通单元中，L16 整体提升 1.015–1.789×；独立的高预算 arXiv / $\alpha = 200$ 对照达到 2.383×。

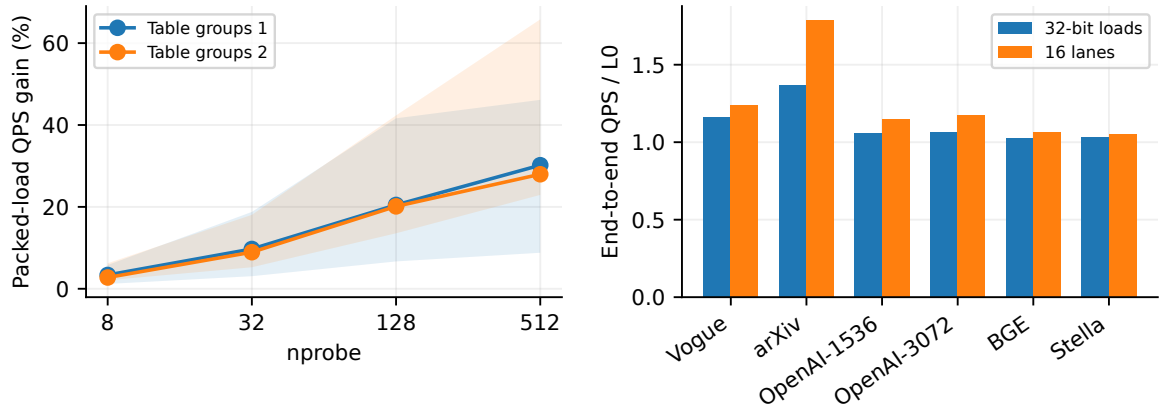


图 7.5. 打包与精排对照。左：各表分组 / probe 数下，六个数据集的打包载入 QPS 收益中位数及最小—最大阴影范围。右：128 probes、各数据集普通预算下，仅改变载入和 16-lane 的实测端到端收益。两个面板使用不同实验家族。

仅改变载入的 L1 与 L0 逐位相同，却在 arXiv 获得 1.368×。在 $\alpha = 100$ 下使用 lanes，精排从 4,277.5→716.8 μs，内核提升 5.97×，而整体为 1.789×。OpenAI-3072 的内核 / 整体提升分别为 8.81× / 1.171×。其他阶段仍存在。

表 7.5. 128 probes 下的精排。比值为相对 L0 的成对端到端收益；末列除外，它是剖析得到的精排内核收益。内核与应用收益不能互换。

数据集	α	$L1/L0$ QPS	$L16/L0$ QPS	内核加速
arXiv	100	1.368	1.789	5.97
BGE	32	1.026	1.062	1.97
OpenAI-1536	8	1.055	1.145	3.34
OpenAI-3072	4	1.063	1.171	8.81
Stella	16	1.030	1.053	2.66
Vogue	64	1.161	1.240	3.72

基线 wall time 占比为 f、精排加速比为 s 时，替换模型给出

$$\frac{T'}{T} = (1-f) + \frac{f}{s}, \qquad \frac{T}{T'} = \frac{1}{1-f+f/s}.$$

arXiv / OpenAI-3072 的 $f = 0.529/0.168$ 预测 1.787/1.175×，实测为 1.789/1.171×。独立剖析提供解释性估计。

Lane FP32 重新结合运算的相对距离误差达到 $\approx 1.6 \times 10^{-6}$，偶尔改变顺序。Sanitizer 只覆盖记录的子集，未声称普遍逐位相同。

7.4 选择正确性与性能

7.4.1 计算分数选择的严格 CPU 参考

T4 在 CPU 上按 $(D_{\mathrm{P}}, \mathrm{position})$ 排序 GPU 生成分数。导出器遵循扫描布局 / 累加顺序，有效返回分数逐位匹配；源码比较发现相对表扫描版本只增加导出。270 个配置覆盖 Vogue / arXiv / OpenAI-3072，probes 1/8/128、$\alpha \in \{4, 100, 200\}$、$g_{\mathrm{scan}} \in \{0, 1, 2, 4, 8\}$ 和两种阈值设置，共 69,120 个实例（表 7.6）。

表 7.6. T4 CPU 参考检查。集合与顺序相等使用完整主码键。有效计数检查 $\min(c, n_c)$ 个保留候选；最终相等按 $(D_{\mathrm{H}}, \mathrm{ID})$ 对保留的计算精排分数排序，包含其 FP32 bits。计数为查询实例，同一查询跨配置复用。

路径	查询数	集合 / 顺序精确	仅并列	有效计数	最终精确
$g_{\mathrm{scan}} \geq 1$	55,296	55,296 / 55,296	0	55,296	55,296
$g_{\mathrm{scan}} = 0$	13,824	13,768 / 8,758	56	13,824	13,824

表 7.6 确认分组完整键顺序与单 block 距离顺序；集合有 56 个纯并列差异。全部 19,120 个不足额计数及 69,120 个保留分数最终选择通过，测试 引理 5.1与命题 5.1与5.2。扫描 / 阈值及 Sanitizer 覆盖仍独立（第3.5.2节）。

7.4.2 共享阈值保持选择目标

48 组输出全部一致，QPS 比为 0.994–1.562×。在 arXiv $\alpha = 200$、batch 1,000、八组下，共享使归并从 2,629→471 μs，扫描从 7,968→5,513 μs。部分单元略有退步。

试验第一组包含 46–81% 的优胜者。相同配额需要回退，不同配额仍留有瓶颈。阈值避免固定配额。

7.4.3 探测列表选择与输出一致性

Radix 保持评分算术 / OpenAI TF32（第7.2.3节）。全部 1,792 个合成用例通过。对 arXiv / Vogue / OpenAI-3072 的全部 1,000 个查询，probes 以及主码分数 / 位置均匹配。

表 7.7. 精确计算分数 probe 改动的 v69 / v65 端到端吞吐比。每个工作负载—batch 比较中，每个二进制有十轮计时，交替运行收集。> 1 的比值有利于 v69。

数据集	Batch 8	32	256	1,000
arXiv	1.074	1.063	1.035	1.034
Vogue	1.109	1.103	1.052	1.041
OpenAI-3072	1.167	1.117	1.055	1.050

表 7.7 显示端到端提升 1.034–1.167×。batch 8 下，剖析的 probe 阶段在 arXiv 从 56.5 降至 22.6 μs，Vogue 从 52.7 降至 14.7 μs，OpenAI-3072 从 77.8 降至 16.7 μs。未改阶段限制全 batch 收益，各比值使用自己的基线。

Vogue 在跨二进制的 1,000 查询转储中，probe / 主码逐字节一致（一组 / batch-8），最终召回却为 0.9642/0.9644。基线自身在一致性 / 计时运行间也由 0.9642 变为 0.9644。本次检查中，变化发生在主码选择之后，原因未解决。同进程内调度 / 单 block 输出匹配（第3.5.1节）。

可选 FP16 probe 集合在 arXiv / Vogue / OpenAI-3072 的 1,000 查询中分别有 936/979/998 个与 v69 一致。高度重合不等于精确。这些结果不纳入主 E1 前沿，也不纳入精确 probe 主张。

7.5 留出预算质量、吞吐与回本

RQ3 将 第6章 的策略与固定 $\alpha = 100$ 及更强的通用对照 $\alpha = 64$ 比较。

7.5.1 无标签选择与有标签诊断 Oracle

六个数据集在 128、512 probes 下测试 第6.4节 的固定、抽样、FULL、枚举和 oracle 对照。不相交查询池及每样本量 64 次抽样遵循 第3.2节。

所有规则变体均使用 式 (6.4)，取 $k = 10$、$\epsilon = 1$。FULL 使用全部校准查询；诊断 oracle 使用留出标签（表 6.1）。

7.5.2 全池校准作为质量优先工作点

每个单元对全部 500 个校准查询执行一次全池校准。所选实验臂在 12/12 单元满足留出 10^{-3} 损失目标，最大观测损失为 0.0006。其速率为 GT oracle 的 0.908–1.000，校准成本 9.5–60.3 ms。相对固定 100 的速率几何平均比为 1.111×；只看固定 100 也达标的十个单元，则为 1.193×。表 7.8 给出完整比较。相对固定 64，全池选择在六个单元更快、两个持平、四个更慢；主要优势是观测中覆盖了全部十二个质量目标。

这些单元支持质量优先时用 FULL，校准成本优先时用 $S = 128$（第6.2节）。

7.5.3 低成本抽样及其质量风险

表 7.9 报告 $S = 128$ 结果。十二个单元中，十个所选预算中位数匹配 oracle；arXiv / 128 probes 和 OpenAI-3072 / 512 不同。预算跨工作负载为 4–200。预算中位数与平均召回不一定对应同一实验臂。平均召回由打印的最佳召回减去平均损失重建，约保留四位小数。

arXiv 在 128/512 probes 下分别有 40/64、32/64 次损失越阈，尽管平均损失只有 0.0009。固定 100 损失 0.0012/0.0016：比默认更好未必达标。BGE / 512 有 24 次越阈且 $L_{95} = 0.0014$；Stella 达到 0.0030；OpenAI-3072 没有越阈（表 7.9）。四次损失可与报告 $L_{95} = 0$ 同时出现（第3.5.2节）。

7.5.4 吞吐估计与校准经济性

按 第3.5.2节 的平均所选臂估计，$S = 128$ 在十个单元超过固定 100，幅度 1.028–1.691×。arXiv 为 0.908/0.917，但平均召回更好，没有吞吐回本。

表 7.8. 全池校准（$S = 500$）。损失相对最佳实测留出召回率；速率为所选固定臂的 kQPS。“Cal.” 是每单元一次全池校准调用，不是重复查询流计时。留出查询为 500 个，Stella 为 499 个。

数据集	Probe	α_{full}	损失	Q_{full}	相对 100	相对 oracle	校准 ms
Vogue	128	100	0.0000	202.9	1.000	0.908	10.1
arXiv	128	200	0.0000	151.5	0.728	1.000	10.0
OpenAI-1536	128	8	0.0000	267.7	1.599	1.000	9.5
OpenAI-3072	128	4	0.0000	145.0	1.691	1.000	17.1
BGE	128	64	0.0006	124.9	1.075	0.932	17.1
Stella	128	16	0.0006	110.4	1.153	1.000	20.6
Vogue	512	100	0.0002	100.1	1.000	0.969	20.9
arXiv	512	200	0.0000	69.0	0.831	1.000	25.1
OpenAI-1536	512	8	0.0000	114.9	1.326	1.000	21.8
OpenAI-3072	512	4	0.0002	52.1	1.219	0.977	40.5
BGE	512	64	0.0006	50.8	1.033	1.000	42.3
Stella	512	16	0.0006	36.0	1.041	1.000	60.3

表 7.9. $S = 128$ 下的留出质量。规则 α 为 64 次抽样中位数；oracle α 使用留出标签。$\overline{R}_{\text{rule}}$ 为重建的平均召回。“Losses” 计数相对最佳已测留出召回损失 $> 10^{-3}$ 的抽样次数；L_{95} 为日志中的较低次序 95 分位损失估计。R_{100} 为固定预算召回。

数据集	Probe	α: 规则/oracle	R_{100}	$\overline{R}_{\text{rule}}$	越阈数	L_{95}
Vogue	128	64 / 64	0.9680	0.9677	5/64	0.0022
arXiv	128	100 / 200	0.9748	0.9751	40/64	0.0024
OpenAI-1536	128	8 / 8	0.9280	0.9279	4/64	0.0000
OpenAI-3072	128	4 / 4	0.9252	0.9252	0/64	0.0000
BGE	128	32 / 32	0.9198	0.9190	1/64	0.0008
Stella	128	16 / 16	0.9892	0.9884	6/64	0.0030
Vogue	512	64 / 64	0.9916	0.9915	2/64	0.0002
arXiv	512	200 / 200	0.9878	0.9885	32/64	0.0016
OpenAI-1536	512	8 / 8	0.9740	0.9738	4/64	0.0000
OpenAI-3072	512	4 / 8	0.9704	0.9702	0/64	0.0002
BGE	512	64 / 64	0.9670	0.9661	24/64	0.0014
Stella	512	16 / 16	0.9954	0.9945	7/64	0.0030

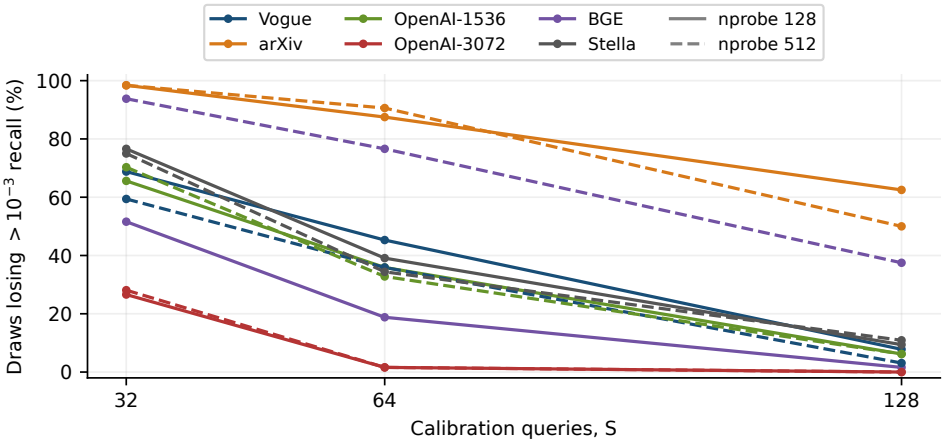


图 7.6. 不同样本量下的留出风险。纵轴为相对最佳已测预算，留出召回损失超过 10^{-3} 的校准抽样百分比。因为 式 (6.4) 保持阈值为一，增加 S 也会收紧允许的槽位不一致比例。

相对固定 64，抽样赢得 9/12 单元：几何平均 1.054×，范围 0.821–1.367×，OpenAI 为 1.052–1.367×。两个固定对照均未满足 arXiv 质量，无法作统一质量匹配比较。

表 7.10. $S = 128$ 下的校准经济性。速率单位为 kQPS；规则速率是平均所选臂 QPS。比值分别比较固定 100 和 64。“Batches” 用 500 查询、Stella 为 499，给出相对固定 100 的回本次数；“Never” 表示无吞吐回本。质量列单独标明固定 100 是否满足留出损失目标。

数据集	Probe	Q_{100}	$\overline{Q}_{\text{rule}}$	相对 100	相对 64	校准 ms	批次数	固定 100 质量
Vogue	128	202.9	224.4	1.106	1.004	4.0	17.0	达标
arXiv	128	207.9	188.8	0.908	0.821	4.2	不回本	未达标
OpenAI-1536	128	167.5	268.5	1.603	1.367	3.9	3.5	达标
OpenAI-3072	128	85.7	145.0	1.691	1.230	5.7	2.4	达标
BGE	128	116.2	131.2	1.130	1.051	6.5	13.2	达标
Stella	128	95.7	110.3	1.153	1.087	7.5	10.8	达标
Vogue	512	100.1	102.9	1.028	0.996	7.5	55.4	达标
arXiv	512	83.1	76.2	0.917	0.870	9.7	不回本	未达标
OpenAI-1536	512	86.6	115.0	1.327	1.268	8.8	6.2	达标
OpenAI-3072	512	42.8	52.1	1.219	1.052	13.1	6.2	达标
BGE	512	49.2	51.1	1.039	1.006	15.8	41.2	达标
Stella	512	34.6	36.0	1.041	1.028	21.3	37.1	达标

校准成本 3.9–21.3 ms。相对固定 100/64，最大正收益预计回本时间为 0.54/2.72 s，FULL 为 1.52/2.25 s（第6.5节、式 (6.15) 与表 7.10）。arXiv 固定对照质量不达标；下面的直接复用同时测成本和召回。

7.5.5 完整决策轨迹与有限池风险

全部 2,304 条抽样曲线和十二条 FULL 曲线均单调，选择与枚举一致。记录的子集和复现 18,432 个网格计数（式 (6.9)）。这验证已记录汇总；任意子集 / 位置不变性及一般的仅距离嵌套仍有条件（第6.3节）。

$S = 128$ 下最大损失为 arXiv / 512 的 0.0050、BGE / 512 的 0.0046，$S = 32$ 下为 0.0294。FULL 保持在 0.0006 内。

arXiv / 128 在 $S = 32/64/128$ 下预测 / 观测越阈分别为 0.965/0.984、0.875/0.875、0.620/0.625（图 7.7）。36 个差距中最大为 BGE / 512 的 0.144，即 0.519/0.375。GT 损失预测需要稳定计数 / 留出臂质量；校准本身不使用标签。

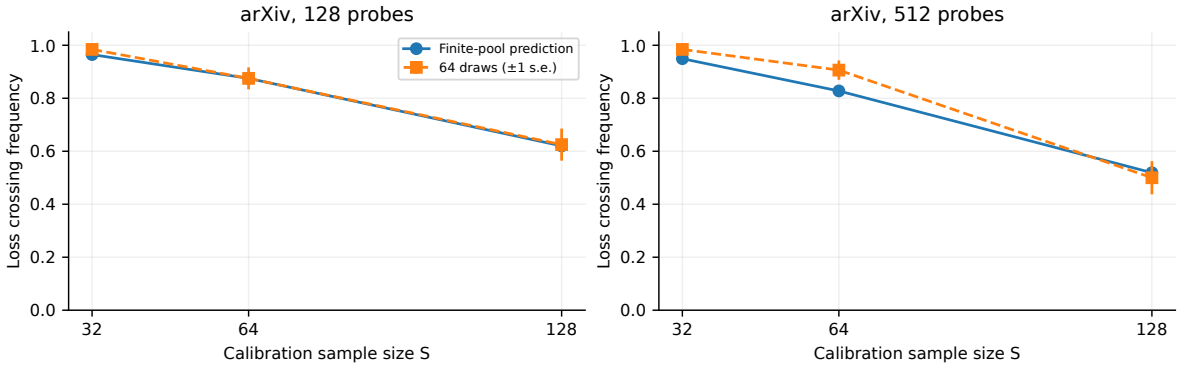


图 7.7. 有限池预测与观测到的 arXiv 损失越阈。误差条为固定池条件下 64 个独立抽样子集的一倍估计标准误，不表示未来工作负载的不确定性。

十次交替重复支持低预算计时平台，而非一般运行时间单调性。

7.5.6 直接测量校准加复用

四个单元在五轮轮换顺序中，对 500 个重复留出 batches 比较固定 64/100、抽样和 FULL。计时包含查询收集、校准、预算应用和首次图，不含构建、预热及扫描校准（表 7.11）。

表 7.11. 500 个复用 batches 的直接总毫秒数，取五轮平均，抽样与 FULL 策略包含预算校准。每 batch 有 500 查询。质量与计时结果一同讨论。

数据集 / probes	固定 100	固定 64	规则 $S = 128$	FULL
arXiv/128	1224	1102	1291	1680
OpenAI-1536/128	1491	1264	852	860
OpenAI-3072/128	2757	2160	1769	1783
BGE/512	5185	5018	4921	5062

OpenAI-3072 两种策略均选 4，召回 0.9252，与固定 100 相同。OpenAI-1536 的 FULL / 固定 100 为 0.9280，抽样为 0.9276。时间见 表 7.11。

相对固定 100/64，直接节省分别从 OpenAI-1536 的 4/6 个 batches、OpenAI-3072 的 4/9 个 batches 开始（图 7.8）；分数批次拟合结果有所不同。BGE 抽样 / 固定 100 / FULL 召回为 0.9658/0.9670/0.9664。arXiv FULL 为质量选择更慢的 200，固定对照不达标。

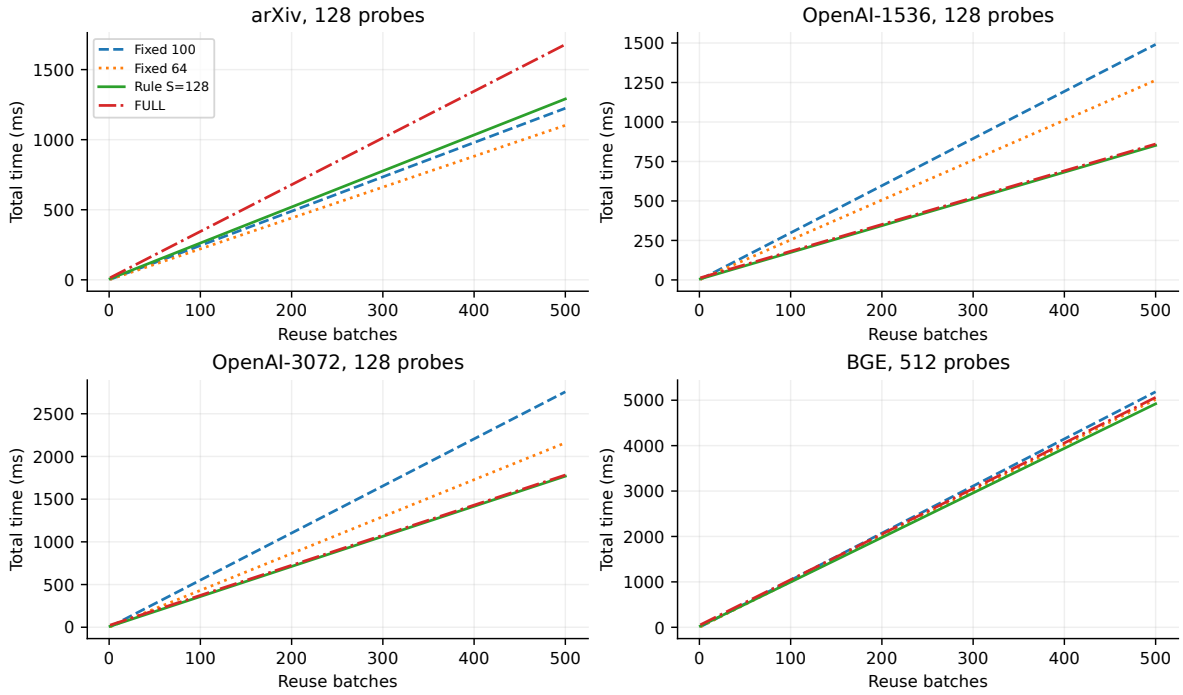


图 7.8. 直接测得的平均累计时间，包含预算校准和首 batch 准备。每条曲线重放一个固定留出查询池。不同策略可能有不同召回率；仅靠时间不能形成质量匹配排名。数据集 / probe 设置与 表 7.11 相同。

7.6 批量大小、调度与负载不均

7.6.1 并行度与重复选择工作

batch-8 的八组提升 2.06–2.76×；arXiv 扫描从 849→216 μs，另加 16.7 μs 归并。batch 1,000、$\alpha = 200$ 时，独立分组反而将扫描从 3,530→7,652 μs，另加 2,624 μs 归并，仅达到单 block QPS 的 0.46×。共享阈值减少工作，但未消除区间变化。

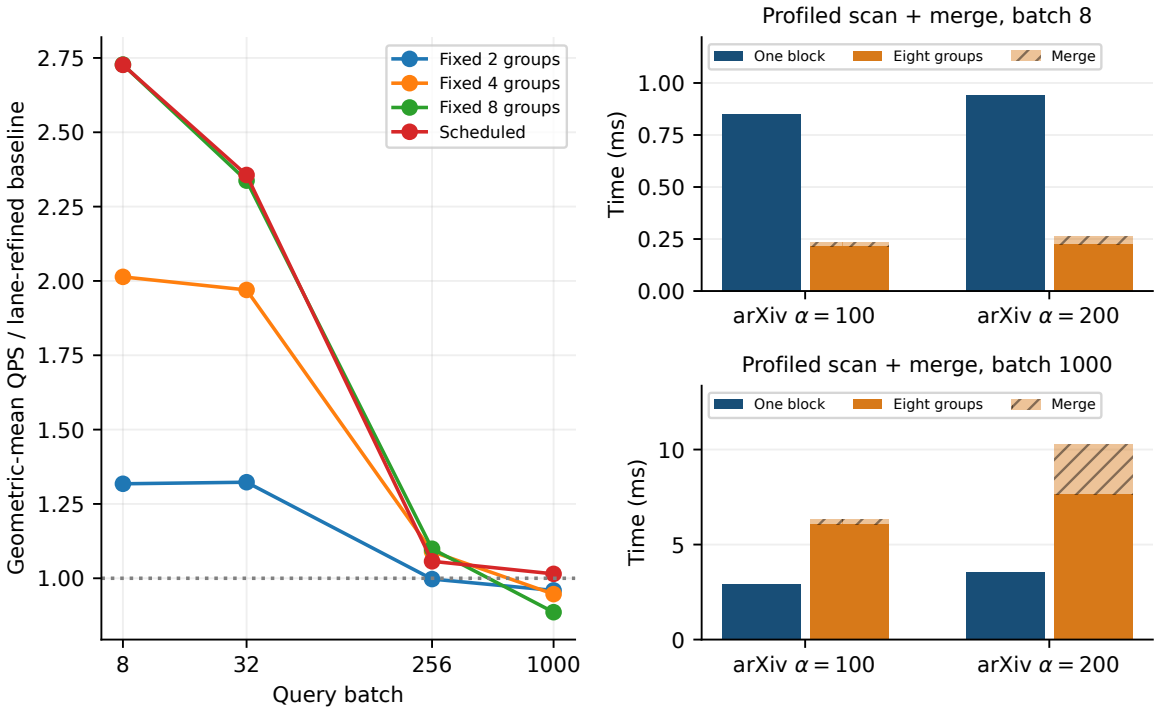


图 7.9. 执行区间。左：固定共享阈值组数及调度执行，相对 lane 精排单 block 基线的速率；小 batch 有 13 单元，batch 1,000 有 11。右：arXiv 预算 100、200 在 batch 8、1,000 下的成对扫描 / 归并剖析。斜纹表示归并。右侧来自独立阈值扫描实验，解释一种成本，不是调度器的完整加速。

7.6.2 在更多配置上评估固定规则

调度遵循 第5.5节 的填充 / 偏斜规则与伪查询校准。四个配置先确定常数，再测量其他配置。

表 7.12. 相对 v61 L16 的集成执行收益。条目为五轮逐单元成对加速比中位数的几何平均。“Held out” 排除确定规则的四个配置，表示留出配置，而非预算实验的留出查询划分。

Batch	单元数	$g_s = 2$	$g_s = 4$	$g_s = 8$	调度	留出配置
8	13	1.318	2.014	2.728	2.727	2.930
32	13	1.323	1.970	2.337	2.356	2.560
256	13	0.997	1.090	1.099	1.057	1.041
1000	11	0.959	0.947	0.886	1.015	1.012

平均值使用十三个小 batch 配置、十一个全 batch 配置；Stella 有 999 查询。留出子集分别为九 / 七个（表 7.12）。batch 1,000 下，八组为 0.886×；最坏 regret 为 1.217（BGE / 256）。

Vogue 的 CV 0.338–0.622 在 batch 1,000 时选择四组。比值范围 0.895–1.175；$(n_{\text{probe}}, \alpha) = (16, 16), (16, 64), (32, 64)$ 对应 0.955、0.895、0.955。CV 测量离散程度，而非总工作量：它使 0.97 处领先，却在低 probe 负载上过度分组（第7.2节）。

250 个变体 / 配置对、234,320 次查询比较中，没有非并列主码差异或最终 ID 集合差异。并列输出顺序可能不同。这些是顺序同步吞吐测试，不是开放到达流尾延迟测试。

7.7 冷构建、常驻显存与已测规模

冷构建、常驻显存和完成的大数据集搜索用于评估 RQ1；E1 显存比较也支持 RQ4。

7.7.1 冷构建与生命周期边界

三次空缓存重复得到 Vogue 训练 298–303 ms，总计 ≈ 0.56 s，而较慢存档为 1.33 s。Stella 为 10.54–10.65 s，存档为 10.40 s（表 7.13）。

表 7.13. 冷 JHQ 构建与单独测量的常驻显存。Train / add 为三次补充正常构建的组件中位数；Total 为成对总和中位数。显存来自 v71 的两组、128 probes、$\alpha = 100$ 运行，包含上下文和工作区。“Residual” 是残差编码模型大小占实测使用总量的比例，不是每查询读取比例。

数据集	训练 s	Add s	合计 s	使用 GiB	原始 GiB	残差占比
Vogue	0.30	0.26	0.56	1.47	2.67	45.4%
arXiv	0.48	0.51	0.99	2.58	6.45	62.5%
OpenAI-1536	0.61	0.38	1.00	2.39	5.72	59.7%
OpenAI-3072	1.16	0.67	1.82	4.14	11.43	69.0%
BGE	3.56	3.65	7.20	11.93	38.50	80.7%
Stella	3.55	6.99	10.55	20.44	67.81	82.9%

第3.3节与第3.5.2节 定义启动 / 构建边界；图 7.10 比较重复构建。

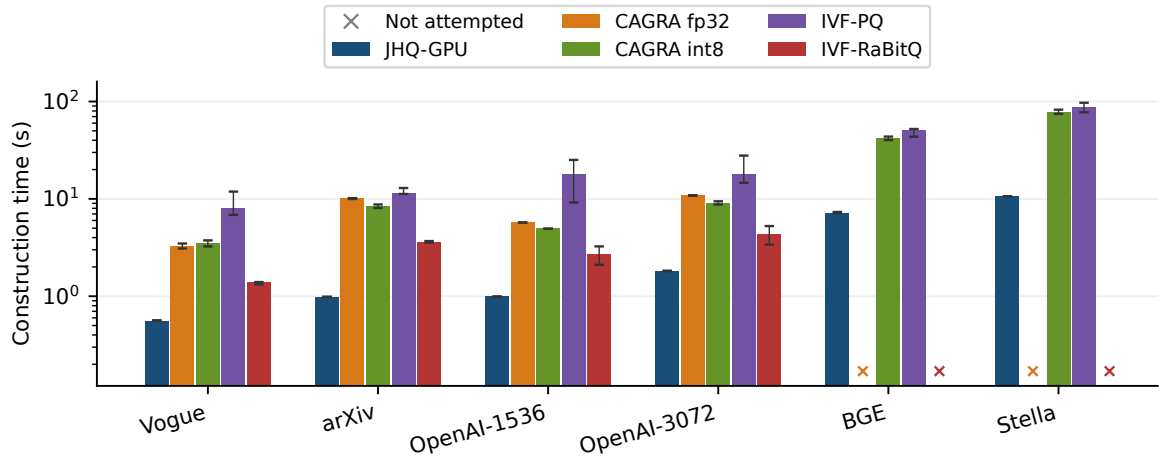


图 7.10. 记录的构建成本，纳入 CAGRA int8。JHQ 为输入页已热时三次补充正常冷构建的中位数及范围；其他柱为成功测量索引配置的中位数，误差线为最小—最大值。Int8 包含数据集量化；RaBitQ 不含所需序列化往返。缺失大数据集柱表示未尝试，不是实测构建失败。纵轴为对数。

Stella 的隔离 CAGRA int8 为 74.55/82.45 s，含量化；三个 IVF-PQ 配置的中位数为 86.59 s，而 JHQ 为 10.54–10.65 s。质量 / 常驻显存另行比较。

五次 arXiv 构建的召回范围为 0.0012。Stella 启动在热页下为 18.2–18.6 s，首次读取为 129.9 s。20 ms 采样峰值为 BGE 26.08 GiB、Stella 22.58 GiB。BGE 的 12 GiB 分配缓冲先于 23.05 GiB gather 载荷存活，不能相加（式 (4.13)）。

7.7.2 常驻分配与压缩的作用

图 7.11 将 第4.4节 组件模型与实测常驻总量结合，保留未归属余量。前述构建测量给出采样峰值。

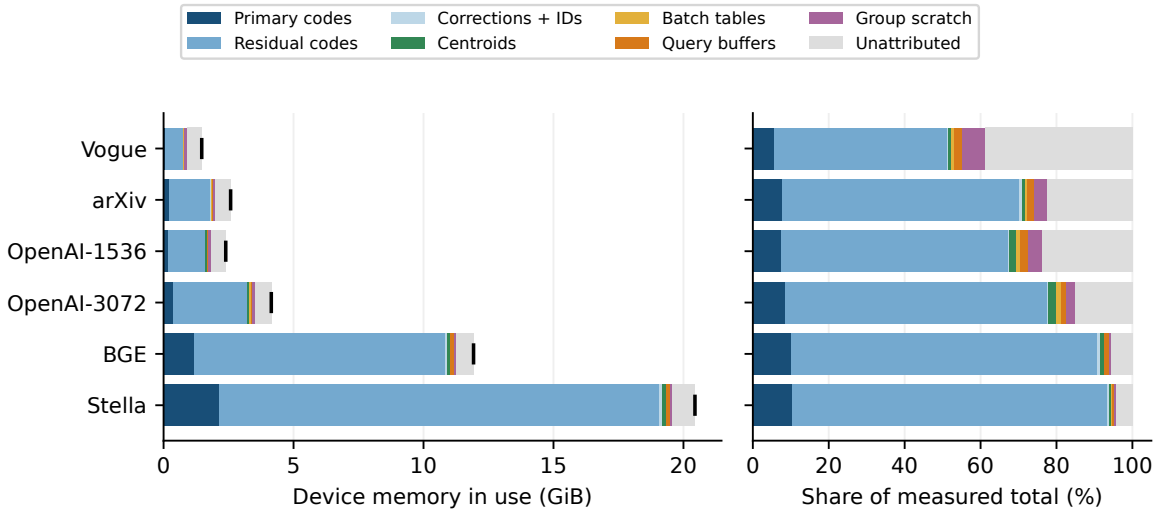


图 7.11. 两组表、128 probes、$\alpha = 100$ 下的常驻显存分解。组件大小为模型值，标记为实测使用总量，斜纹为未归属余量。总量包含上下文和工作区。前述构建测量给出采样峰值；这些常驻标记不是峰值。

残差占 BGE 11.93 GiB 总量的 80.7%，占 Stella 20.44 GiB 的 82.9%，总量含上下文 / 工作区。原向量需要 38.50/67.81 GiB。更小 α 节省读取，不减少存储残差 bytes（第4.4节）。

E1 测量在 表 7.14 提供独立跨系统视角。从 JHQ 进程总量减去基线测得的 507.2 MiB 上下文，约为 CAGRA fp32 索引分配的 0.29–0.39。JHQ 包含工作区，CAGRA 则为构建时索引增量。这些是参数扫描范围，不是显存匹配召回点或等峰值预算比较。同表 CAGRA int8 与 JHQ 分配同量级：相对 fp32 的压缩并非 JHQ 独有。表 4.1 组件模型解释这一比较。

表 7.14. E1 分配范围。JHQ 为进程总量减去估计的 507.2 MiB 上下文，包含查询工作区；两个 CAGRA 列是构建时测得的索引分配。范围覆盖参数扫描。

数据集	JHQ 净值进程 GiB	CAGRA fp32 索引 GiB	CAGRA int8 索引 GiB
Vogue	0.90–1.01	2.89–2.91	0.89–0.91
arXiv	2.01–2.19	6.84–6.98	2.01–2.15
OpenAI-1536	1.79–2.33	5.95–5.97	1.67–1.68
OpenAI-3072	3.54–3.82	11.67–11.68	3.10–3.11

7.7.3 大数据集可行性与竞争区间

BGE 包含 10,091,524 个向量，Stella 为 17,776,615；二者均为 1,024 维、32,768 个 IVF 列表。Stella 查询从语料中抽取，因此结果刻画这一基准，不是独立采集的外部查询分布。图 7.12 纳入成功的 CAGRA int8 与 IVF-PQ 测量。预计存储超容量时，完整精度 CAGRA / RaBitQ / FP32 暴力搜索被预先排除。这是计划排除，不是实测 OOM 或召回失败。

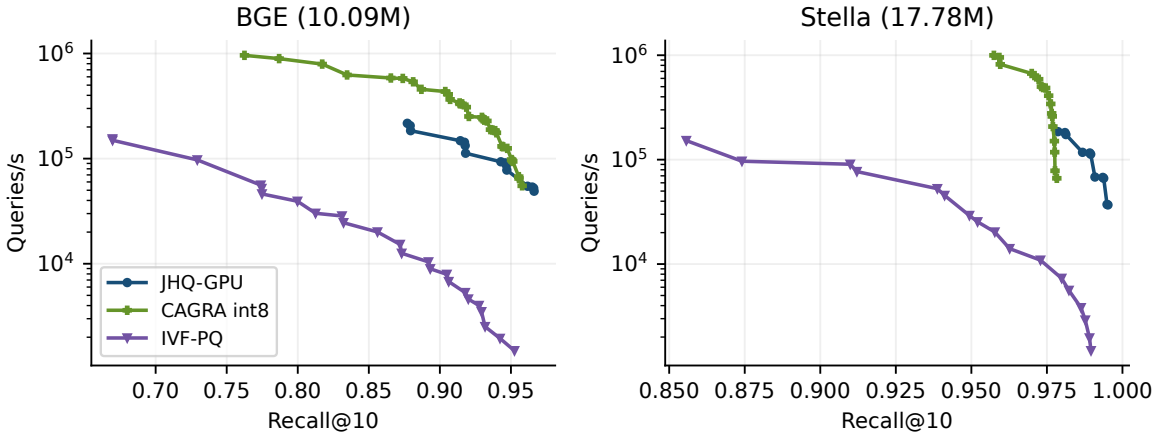


图 7.12. 大数据集主机到主机前沿，纳入成功的隔离 CAGRA int8 重跑。BGE / Stella 使用 1,000/999 查询。端点标记已测网格覆盖。

CAGRA int8 可容纳 Stella。在 BGE 召回率 0.88–0.95 上，它领先 1.35–2.96×；在 int8 最高召回率 0.9579 处，JHQ 领先 1.086×。BGE / Stella 最高召回分别为：int8 0.9579/0.9783，JHQ 0.9663/0.9952，PQ 0.9525/0.9896。

Stella 上，JHQ 在召回 0.978679 时为 185.4 kQPS，int8 在 0.978278 时为 66.3。2.80× 的测点比并未匹配召回；int8 不覆盖更高区间（图 7.12）。

7.8 数值检查与召回分析

7.8.1 质量分辨率与重复执行

一个 GT 命中使 Recall@10 改变

$$\Delta r = \frac{1}{nk} = \begin{cases} 10^{-4}, & n = 1000, \\ 2 \times 10^{-4}, & n = 500. \end{cases}$$

Stella 的 $n = 499$ 步长略大。这个步长不是确定区间。

全部 369 个 E1 基线重复 ID 标记均保留，包括并列重排。401 口径增加跨协议差异（第3.5.2节）；这些不是独立构建。

0.97 附近，吸附前的局部斜率对应每 10^{-3} 召回变化 1.01–3.17% QPS。五次构建的召回范围在 Vogue / arXiv / OpenAI-1536/3072 上为 0.0008/0.0012/0.0004/0.0002。这提示局部敏感性，不是重新训练前沿的界。缓存模型 FP32 检查保持 OpenAI 优势（第7.2.3节）；arXiv / LUT 的接近持平仍无定论。

RaBitQ 在 Vogue / arXiv / OpenAI-1536/3072 上，五轮范围占中位 QPS 的中位数为 3.79%、3.34%、2.87%、5.25%。这些离散范围不是置信区间。为降低偶发 30–60 ms 调用的影响，本文将 1,000 查询速率重算为 $10^6/\,\mathrm{median(call\ ms)}$。召回率 0.97 时，所得 JHQ / RaBitQ 比为 1.063、0.995、1.118、1.100。三个有利方向和 arXiv 接近持平仍保持，因此结果并非仅由长调用造成。

7.8.2 重建评分与召回损失

表 5.1 的阶段约束区分计算分数选择与原始空间 kNN；lane 舍入、cutoff 并列和进程变化分别限定不同阶段。

存档重建试验测量距全 probe 重建评分的剩余差距。七个工作负载—预算臂各有 128 查询，对全部 probed 候选做 FP64 评分，相比所选预算的 GPU 输出，召回提高 0–0.002344。arXiv / 128 probes 下，$\alpha = 200$ 在全部 128 查询上匹配全 IVF 重建答案集。预算为 100 时，召回 0.9734375，全 IVF 为 0.9750；在路由 / 重建共同缺口 0.0250 之外，还有 0.0015625 的预算缺口（第6.2节）。这些汇总包含同一数据集—probe 工作负载的两个预算。FP64 / GPU 算术差异使部分实验臂不能纯粹归因为预算。分离路由 / 表示需要逐阶段 GT 覆盖。

第 8 章讨论与未来工作

8.1 设计启示

8.1.1 码本结构与查表开销

Mailman 是一种复用有限字母表编码中间和的方法，在这里并未获得速度收益；两组 LUT 通常获胜（图 7.4）。因此，码本质量必须结合可实现的表放置 / 访问成本评估（第5.2节与第5.3节）。

8.1.2 并行度与选择开销

当查询已提供足够并行度，暂存 / 选择 / 归并开销可能主导（图 7.9）。Vogue / BGE 暴露了调度缺少绝对工作量 / 分解特征（第7.2节与第7.6节）。更复杂调度必须收回校准成本；共享阈值的独立作用尚未分离。

8.1.3 精排速度与预算收益

假设固定 $T_0 > 0$、精排时间 $F_h \geq F_\ell > 0$ 及共同倍率 $\eta \in (0, 1]$。低预算加速比及导数为

$$S(\eta, T_0) = \frac{T_0 + \eta F_h}{T_0 + \eta F_\ell}, \tag{8.1}$$

$$\frac{\partial S}{\partial \eta} = \frac{T_0(F_h - F_\ell)}{(T_0 + \eta F_\ell)^2} \geq 0, \qquad \frac{\partial S}{\partial T_0} = \frac{\eta(F_\ell - F_h)}{(T_0 + \eta F_\ell)^2} \leq 0.$$

当 $F_h > F_\ell$ 时，更快的精排降低削减预算的价值，而更低固定成本提高这一价值。导数成立的前提是非精排工作固定。依赖预算的扫描维护和资源转换需要 第7.5节与第7.5.6节 的受控比较。质量和复用仍是约束（表 7.8与图 7.6）。

8.2 替代评分与编码实验

8.2.1 Mailman 式共享和评分

Mailman 复用有限字母表求和 [20]。固定的 Vogue / arXiv / OpenAI-3072 流比较宽度 4/6/8 的原子桶 / 私有直方图和打包双表评分：128 probes、batch 8/1,000，编码 / 数学分数固定。

最佳替代仅达到基线 QPS 的 0.187–0.581（图 8.1）。偏斜的桶目标主导剖析；私有直方图增加状态 / 共享更新。更窄分组有时有益，但没有获胜方案。其他有限字母表设计仍值得研究。

测试仅覆盖分数，因此 cutoff 检查不证明近邻流水线等价。替换模型给出

$$\frac{T'}{T} = 1 + f\left(\frac{1}{s} - 1\right) > 1 \quad (0 < f \leq 1,\ 0 < s < 1).$$

该模型不是实测端到端性能，集成已停止。字典研究也停在结构统计。二者都不产生系统前沿测点。

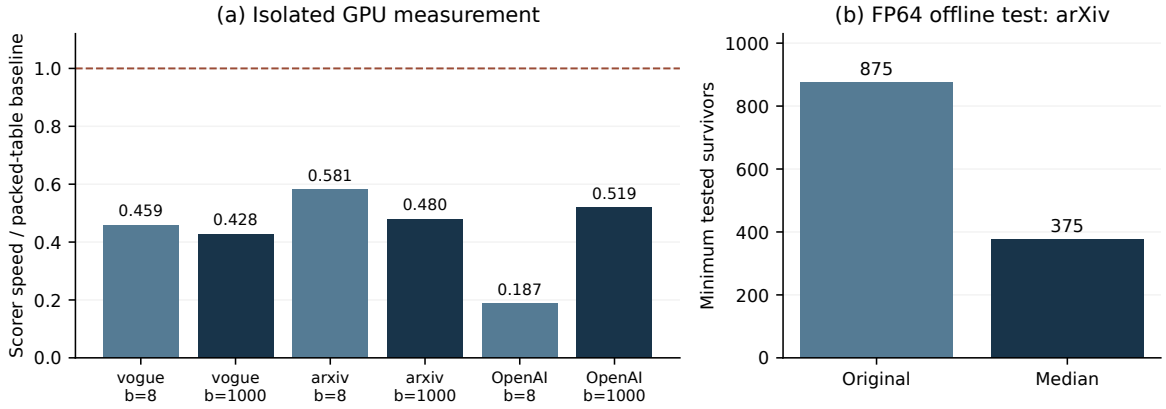


图 8.1. 两个不同的证据边界。左：RTX 5090 上，Vogue、arXiv 和 OpenAI-3072、两个 batch、128 probes 时，最佳实测 Mailman 评分器 / 打包表评分器速度比，全部小于一。右：arXiv 400 查询 FP64 离线测试划分上，达到原 JHQ 召回目标的最小保留数。候选减少不是 GPU 加速。

8.2.2 面向排序的编码实验

FP64 候选重放使用 400/200/400 个校准 / 验证 / 测试查询，测试加权分数、中位数 / 列表阈值、学习式损失和线性最小均方误差（minimum mean-square error，MMSE）解码。更低 MSE 可能恶化近邻顺序；主码等级变化也会移动残差参考点。

中位阈值将 arXiv 达标的最小已测保留数从 875→375，但另两个工作负载失败。对正交 Π，

$$\bar{y} = \frac{1}{N}\sum_i \Pi x_i = \Pi \bar{x}, \qquad \|\bar{y}\|^2 = \bar{x}^\top \Pi^\top \Pi \bar{x} = \|\bar{x}\|^2.$$

因此旋转保持均值向量范数，而不消除偏置。中心化也必须保持精排分数上限。

中位等级 GPU 后续实验重新训练残差。对 arXiv 原 $\alpha = 100$ 目标，batch 8/1,000 下提升 1.40/1.21×；Vogue 仅达到 0.9630，低于 0.9644；OpenAI-3072 持平。三轮使用独立状态 / 查询，不属于 E1。

8.3 内部与外部有效性的威胁

表 8.1 汇总主要解释的有效性威胁。对照见第 3 章，重复 / 构建 / 精度检查见 第7.8节。

表 8.1. 可能逆转结论或缩小适用范围的威胁。

受威胁主张	替代解释或反转	对照与剩余测试
执行收益	分数、并列或索引状态变化解释表观加速	严格 T4 CPU 排序与表形式诊断；覆盖有限，候选转储已不保留
预算质量	有限池抽样或漂移隐藏有害查询	不相交 GT 评估、完整轨迹、子集和检查及 FULL 枚举；未来查询和任意子集不变性未测试
竞争优势	网格密度、调用尾部或 API 开销改变排序	两种召回估计器、保留慢调用及 int8 基线；TF32 / FP32 敏感性；T5 暴露小 batch 反转；跨设备未测
容量 / 构建主张	瞬时分配超过常驻采样	阶段字节、重复空缓存构建和 20 ms 设备峰值；采样间分配及完整重训练前沿未测

历史 CPU JHQ 测试与 E1 的索引参数、召回率口径和计时协议不同（第7.1节）。这些吞吐数字不能构成相同质量下的 CPU / GPU 加速比；该对照仍未测量。

证据覆盖一张 GPU、六个准备后的数据集和平方 L2 top-10（第7.2.6节）。可能仍有近重复，没有相关性标签。更新、并发、漂移、服务延迟、十亿规模和多设备均未测试。

8.4 未来研究方向

8.4.1 校准风险与复用成本

未来成本感知校准需要有效的查询级界、匹配执行区间的计数、固定参考、抽样 / 排序假设及漂移更新。绝对 GT 风险还需要标签，或有根据的参考到 GT 联系。

可通过留出总时间和尾部一致性，比较抽样、排名复用和成本感知选择。四单元顺序控制器因额外启动而落败。替代方案必须满足风险界，并超过质量匹配对照。

8.4.2 GPU 与主机内存间的残差存储

未来残差缓存可在设备容量上限下获取主机数据块。固定表示、路由、保留集合和算术，改变容量、batch 和局部性，测量字节数、命中、重叠、尾部和 QPS。这项容量研究尚未实现；降低 α 并不会迁移已存残差。

8.5 部署建议

召回率 0.97 时，E1 支持 batch-1,000 吞吐结论；batch 8 下 T5 有利于 CAGRA fp32。batch-32 的 OpenAI-3072 取决于估计器（第7.2.6节）。可以从 FULL 开始，再验证质量、复用成本和显存（第8.3节）。

第 9 章结论

9.1 对四个研究问题的回答

RQ1：GPU 索引构建与容量。 GPU 构建生成可搜索的常驻层级。最大测试容纳 17.78 百万个向量，常驻 20.44 GiB，构建耗时 10.54–10.65 s；构建峰值见 第7.7节。

RQ2：面向表示结构的执行。 Cartesian 拆分保持实数算术分数；完整键保留 / 归并和见证阈值在其前提下保持分组选择。两组 LUT 在 22/24 单元获胜。调度在 batch 8/1,000 下相对 lane 精排提升 2.727/1.015×（第7.3节与第7.6节）。严格 CPU 检查确认分组顺序；单 block 差异均为 cutoff 并列（第7.4.1节）。

RQ3：工作负载级精排校准。 FULL 在 12/12 单元满足已测试的相对召回损失目标，最大观测损失 0.0006。$S = 128$ 在一个单元有 40/64 次越阈（第7.5节）。FULL 与枚举一致；十个仅距离单元缺少一般嵌套保证。参考一致性不是总体 GT 召回率。四项直接复用测试显示质量 / 时间收益依赖工作负载。

RQ4：跨系统搜索性能。 batch 1,000 / 召回率 0.97 时，JHQ 在三个数据集领先，在 arXiv 与 RaBitQ 接近持平；Vogue 的领先依赖分组（第7.2节）。OpenAI E1 使用 TF32。在 TF32 训练缓存上，FP32 检查保持有利方向（第7.2.4节）。Int8 / RaBitQ 在其他召回区间获胜；CAGRA fp32 赢得全部 batch-8 数据集。batch-32 接近持平时依赖估计器（第7.2.6节）。

9.2 总结与展望

JHQ 的 Cartesian 结构需要由执行路径将小表和打包编码转化为低成本访问。并行分解的收益随后取决于工作负载：分组为小 batch 暴露更多工作，而大 batch 本身可能已有足够多的独立查询。精排校准还引入了单独的质量决策，因为有限查询池上的一致性不能保证未来召回率。下一步应把风险受控的校准与实测复用成本结合，再研究显存成为主要约束时的残差放置（第8.4节）。

参考文献

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, 2015. doi: 10.14778/2856318.2856324.

[2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43(5):1666–1677, 2021. doi: 10.1109/TPAMI.2019.2952606.

[3] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2022. URL `https://arxiv.org/abs/2110.01052v5`. Version 5; Section 2.3.1.

[4] Anthropic. Claude: Implementation, experiment and review assistance record. Project AI-use record, 2026. Implementation and experiment records; original prompts and interaction metadata unavailable.

[5] Tony Assi. Vogue933k embeddings. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/tonyassi/vogue933k-embeddings`. Upstream attribution recorded in the project data preparation source.

[6] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data*, 3 (4):242:1–242:26, 2025. doi: 10.1145/3749160. URL `https://arxiv.org/abs/2505.19001`.

[7] Artem Chirkin, Akira Naruse, Tamás Fehér, Yong Wang, and Corey Nolet. Accelerating vector search: NVIDIA cuVS IVF-PQ part 2, performance tuning, 2024. URL `https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-performance-tuning-part-2/`. 18 July; accessed 28 October 2026.

[8] FAISS contributors. FAISS clustering parameters, version 1.9.0. `https://github.com/facebookresearch/faiss/blob/v1.9.0/faiss/Clustering.h`, 2024. Default minimum-points warning threshold; accessed 28 October 2026.

[9] FAISS contributors. *Index I/O, Cloning and Hyper-parameter Tuning*, n.d.. URL `https://github.com/facebookresearch/faiss/wiki/Index-IO,-cloning-and-hyper-parameter-tuning`. Auto-tuning the runtime parameters; accessed 28 October 2026.

[10] FAISS contributors. *IndexRefine: Candidate Expansion and Refinement*, n.d.. URL `https://faiss.ai/cpp_api/file/IndexRefine_8h.html`. C++ API documentation; accessed 28 October 2026.

[11] Jianyang Gao and Cheng Long. RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2405.12497`. SIGMOD 2024 paper.

[12] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2409.09913`.

[13] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization for approximate nearest neighbor search. In *IEEE CVPR*, pages 2946–2953, 2013. doi: 10.1109/CVPR.2013.379. URL `https://openaccess.thecvf.com/content_cvpr_2013/html/Ge_Optimized_Product_Quantization_2013_CVPR_paper.html`.

[14] Oded Green, Robert McColl, and David A. Bader. GPU Merge Path: A GPU merging algorithm. In *Proceedings of the 26th ACM International Conference on Supercomputing*, pages

331–340, 2012. doi: 10.1145/2304576.2304621. URL https://davidbader.net/publication/2012-gm-ba/2012-gm-ba.pdf.

[15] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896, 2020. URL https://proceedings.mlr.press/v119/guo20h.html.

[16] Jiabao Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[17] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[18] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. URL https://arxiv.org/abs/1702.08734.

[19] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020. doi: 10.1145/3318464.3380600. URL https://www.pdl.cmu.edu/PDL-FTP/BigLearning/mod0246-liA.pdf.

[20] Edo Liberty and Steven W. Zucker. The Mailman algorithm: A note on matrix–vector multiplication. *Information Processing Letters*, 109(3):179–182, 2009. doi: 10.1016/j.ipl.2008.09.028. URL https://www.cs.yale.edu/publications/techreports/tr1402.pdf.

[21] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. arXiv:1603.09320, 2016. URL https://arxiv.org/abs/1603.09320. Version 4, revised August 2018.

[22] National Institute of Standards and Technology. *Dataplot: Exact Binomial Confidence Limits*, n.d.. URL https://itl.nist.gov/div898/software/dataplot/refman2/auxillar/exacbino.htm. One-sided exact binomial limits; accessed 28 October 2026.

[23] National Institute of Standards and Technology. *Dataplot: Hypergeometric Probability Density Function*, n.d.. URL https://www.itl.nist.gov/div898/software/dataplot/refman2/ch8/hyppdf.pdf. Sampling without replacement; accessed 28 October 2026.

[24] NVIDIA. *NVIDIA Blackwell Tuning Guide*, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/blackwell-tuning-guide/index.html. Section 1.4.1.1; accessed 28 October 2026.

[25] NVIDIA. *CUDA C++ Programming Guide, Release 13.0.1*. NVIDIA, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/cuda-c-programming-guide/index.html. Sections 10.14.1.4 and 18.5.3.3; accessed 27 October 2026.

[26] NVIDIA. *PTX ISA, Version 9.0*, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/parallel-thread-execution/index.html. Sections 8.1, 8.4.2, 8.7 and 8.10.3; accessed 28 October 2026.

[27] NVIDIA. *CUDA Programming Guide: The CUDA Platform*, 2026. URL https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/cuda-platform.html.

PTX compatibility and just-in-time compilation; accessed 28 October 2026.

[28] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[29] OpenAI. ChatGPT: Project review assistance record. Project AI-use record, 2026. Retained review excerpts; original prompts and interaction metadata unavailable.

[30] OpenAI. Codex: Selected thesis-preparation interactions. Project AI-use record, 2026. Selected prompts and uses; compiled 29 October 2026. Partial record; historical interaction metadata unavailable.

[31] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 1536 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-1536-1M`. Accessed 28 October 2026.

[32] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 3072 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-3072-1M`. Accessed 28 October 2026.

[33] Jifan Shi, Jianyang Gao, James Xia, Tamás Béla Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

[34] Suhas Jayaram Subramanya, Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL `https://proceedings.neurips.cc/paper_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html`.

[35] The Alexandria Index. arXiv abstracts dataset. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/macrocosm/arxiv_abstracts`. Accessed 28 October 2026; exported embedding preparation is project-specific.

[36] The Information Engineering Lab. Stella TREC24 BioGen embedding. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/ielabgroup/stella_trec24_biogen_embedding`. Corpus split; accessed 28 October 2026.

[37] Upstash. Wikipedia 2024-06 BGE-M3. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/Upstash/wikipedia-2024-06-bge-m3`. Italian configuration; accessed 28 October 2026.

[38] Karthik Venkatasubba, Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. BANG: Billion-scale approximate nearest neighbour search using a single GPU. *IEEE Transactions on Big Data*, 11(6):3142–3157, 2025. doi: 10.1109/TBDATA.2025.3581085. URL `https://ssomesh.github.io/docs/papers/tbd25.pdf`.

[39] Jingrong Zhang, Akira Naruse, Xipeng Li, and Yong Wang. Parallel top-k algorithms on GPU: A comprehensive study and new methods. In *SC '23*, 2023. doi: 10.1145/3581784.3607062. URL `https://sc23.supercomputing.org/proceedings/tech_paper/tech_paper_pages/pap294.html`.

[40] Weijie Zhao, Shulong Tan, and Ping Li. SONG: Approximate nearest neighbor search on GPU. In *IEEE ICDE*, pages 1033–1044, 2020. doi: 10.1109/ICDE48307.2020.00094. URL `https://www.cs.rit.edu/~wjz/papers/conference/2020-icde-song.pdf`.